Highlights

Handling Conflicts in Microscopic Railway Timetabling At Scale: A Hybrid Approach

Florian Fuchs, Thomas Dubach, Jan Lordieck, Francesco Corman, Bernardo Martin-Iradi

- A hybrid approach combining Logic-Based Benders and Mixed-Integer Programming.
- A network-based model strengthened by advanced reduction techniques and tight bounds.
- New methods for train timetabling exploiting decomposition and relaxation approaches.
- Two complementary approaches leverage network topology differently to improve global scalability.
- Verified on the DISPLIB 2025 benchmark, reaching 81 % of best-known solutions.

Handling Conflicts in Microscopic Railway Timetabling At Scale: A Hybrid Approach

Florian Fuchs[a,b], Thomas Dubach[a,b], Jan Lordieck[a], Francesco Corman[a], Bernardo Martin-Iradi[a]

[a]IVT Transport Systems Group, ETH Zurich, Zurich, 8093, Switzerland
[b]equal contribution

Abstract

This paper presents a hybrid optimization framework for microscopic railway timetabling that integrates Logic-Based Benders Decomposition with Conflict Discovery (`CD-LBBD`) and a Soft-Conflict Mixed-Integer Programming (`SC-MIP`) formulation. The `CD-LBBD` method separates high-level coordination from microscopic feasibility, employing an SMT-based subproblem to dynamically detect and resolve resource conflicts. The `SC-MIP` formulation, by contrast, introduces a continuous relaxation of disjunctive precedence constraints and iteratively minimizes conflict violations to obtain feasible schedules that are subsequently refined to optimality. Both approaches operate on a strengthened continuous-time event–activity formulation enhanced by bound propagation, structural clustering, traversal bounds, and resource aggregation into train-specific chunks.

The proposed framework is evaluated on the complete DISPLIB 2025 benchmark set, comprising 112 real-world–based instances. Within a 10-minute competition limit, `CD-LBBD` and `SC-MIP` jointly solved 65 instances to optimality. The hybrid solver achieves 91 best-known solutions—approximately 81% of the benchmark—and obtained the highest overall score in the DISPLIB 2025 competition. These results demonstrate that combining exact methods with complementary algorithmic structures provides robust and scalable performance for large-scale, microscopic railway timetabling.

Keywords: Microscopic Railway Timetabling, Logic-Based Benders Decomposition, Mixed-Integer Programming, Conflict Resolution

1. Introduction

Timetables translate railway infrastructure into usable capacity by planning train services. They determine operational efficiency and supply quality, both of which are central to the success of railway operations. However, its scope and process integration vary depending on the planning culture. In European networks, timetabling is not an isolated stage in the general planning process but rather recurs across the entire planning hierarchy—from long-term strategic design to detailed tactical derivation, short-term operational updates, and real-time dispatching (Caimi et al., 2017). This paper provides a process sketch of the typical European industry process. Each stage requires resolving the same fundamental task—assigning feasible times, routes, and orderings to resource-constrained train movements—albeit under different scopes, assumptions, and response times. In North American freight railways, by contrast, timetable decisions are often made in a more decentralized and ad hoc manner, with a focus on individual train movements rather than globally coordinated periodic plans (Kloster et al., 2025). Despite these differences in scope and institutional context, both approaches ultimately solve variants of the same core problem: assigning conflict-free times, routes, and orderings to train movements over shared infrastructure, thereby optimizing a given objective (e.g., train delay, passenger travel time).

This paper addresses the microscopic variant of the *train timetabling problem* (TTP) on a tactical or operational level, where infrastructure is represented at track-segment resolution and trains may follow alternative routes, but have at least rough time slots for their operations assigned. In this setting, timing, ordering, and routing are tightly coupled, and feasibility requires a decision over precedences, routes, and timings across interacting trains. The resulting optimization models quickly increase in size and complexity for larger networks and denser services, presenting many variables and conflicts. Solving them to optimality within practical time limits requires decomposition and strengthening techniques that exploit problem structure (Cacchiani et al., 2014).

We propose a hybrid solution method that leverages a tight network-based representation of the problem and combines large-scale decomposition methods (i.e., Logic-Based Benders Decomposition (LBBD)) with a strengthened Mixed-Integer Programming (MIP) model.

We evaluate our approach on the real-world–based instances of the DISPLIB 2025 benchmark set (Kloster et al., 2025). These instances cover a wide range of routing flexibility, network topologies, and train densities. Our approach shows increased performance and scalability, finding optimal or near-optimal solutions to most instances in relatively short computational times. This confirms that carefully hybridized exact methods can address realistic microscopic timetabling problems with strong guarantees under tight computational budgets.

Specifically, our contributions to the existing body of literature are:

- A Event-Activity-Network-based TTP model strengthened by advanced variable and constraint reduction techniques as well as implied bounds.
- A MIP-based approach that iteratively refines time window conflicts to guide the solution search first to feasibility and then to optimality.
- A LBBD framework using a subproblem solved with Satisfiability Modulo Theories (SMT), enhanced with a dynamic conflict discovery procedure.
- A hybrid solver architecture that exploits complementary strengths of MIP and LBBD, enabling robust performance across diverse instance types.
- A systematic evaluation on the full DISPLIB 2025 benchmark library.

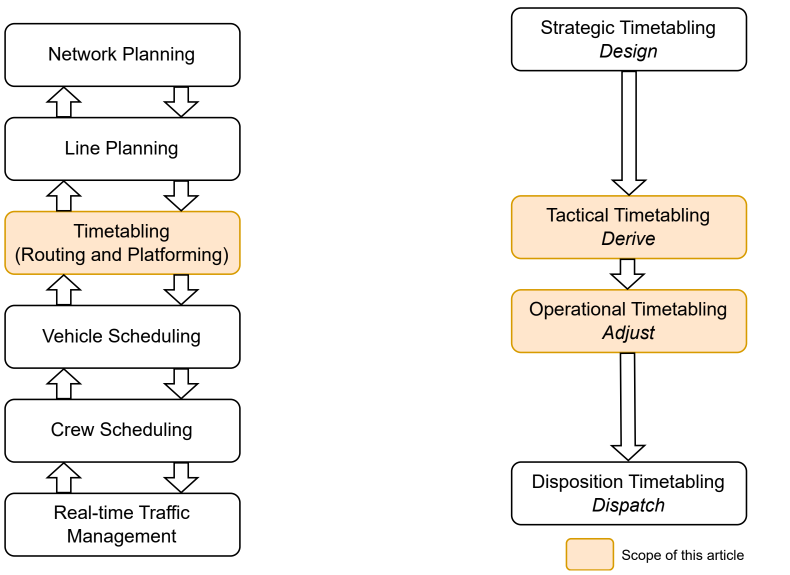
The remainder of the paper is structured as follows. Section 2 reviews related work on microscopic timetabling. Section 3 introduces the TTP with the event-activity-based formulation. The model strengthening and reduction techniques are presented in Section 4 followed by the full model formulation in Section 5. Both the MIP-based and LBBD-based methods are described in Section 6 followed by Section 7 reporting computational results on the DISPLIB benchmark set. We conclude in Section 8 with a discussion of limitations and future directions.

2. Background and Literature Review

This section introduces railway timetabling on a microscopic level. We begin by discussing how timetabling problems can be classified along standard industry processes. We review direct optimization approaches that jointly optimize routing, ordering, and timing, as well as decomposition methods designed to address scalability in high-fidelity models.

2.1. Railway Operation Planning: Timetabling and Dispatching

The scientific literature on railway operation planning often refers to the sequential model introduced initially by Bussieck et al. (1997) and popularized by Liebchen and Möhring (2007) (see Figure 1a). Timetabling is located sequentially between line planning and vehicle scheduling. This view stems from the traditional formulation of the Periodic-Event-Scheduling-Problem (PESP), where routes and frequencies are given, and the objective is typically to schedule trains to minimize travel time. Although this process model includes feedback loops, it overlooks the actual planning procedure employed by some European railways, which utilize a timetable of increasing granularity throughout the entire process (Caimi et al., 2017; SBB Infrastruktur / SBB Cargo, 2014). To position our approach within established industry processes, we introduce this procedure in Figure 1b, alongside the process model commonly referred to in the scientific literature.



(a) Sequential planning procedure

(b) Increasing granularity planning procedure

Figure 1: Railway operation planning process. The planning stages of Subfigure 1a are defined by solving one task of railway operation planning; the whole process is sequential with some feedback loops (based on Bussieck et al. (1997); Liebchen and Möhring (2007); Fuchs et al. (2022). The planning stages of Subfigure 1b can instead always include all tasks for operation planning by making a timetable at different granularity levels. This model is described in text by Caimi et al. (2017) and by industry in SBB Infrastruktur / SBB Cargo (2014)

Strategic timetabling provides the basis to derive a tactical timetable. It is usually interconnected with line planning, where stop sequences, transfer connections, and macroscopic routes of train lines are defined, typically using a macroscopic infrastructure model. In macroscopic models, the physical infrastructure is aggregated into larger segments, for example, between stations. Minimum headways usually model track occupation. In terms of scheduling, strategic timetabling can define arrival or departure time intervals for trains in major stations; in the

European context, this typically involves a standard hour of operation that repeats periodically throughout the day. Sometimes it simply defines the frequencies of lines, which can, however, be translated into repeated time slots throughout the day. It typically supports the planning of future infrastructure expansion or the development of new supply strategies for railway services. (Caimi et al., 2017)

The approach presented in this paper requires stop sequences and time slots per train, defining rough intervals for at least one event, making it suitable for tactical and operational timetabling tasks. Tactical timetabling refers to the process of deriving a detailed timetable from a strategic timetable. Its task is to assign feasible time slots, microscopic routes, and orders to train movements over a microscopic infrastructure network, subject to operational and technical constraints (Liebchen and Möhring, 2007; Peeters, 2003). Microscopic, in this case, refers to using a detailed infrastructure model that contains every operation-relevant track section, switch, platform, and train control system object. This allows for realistic track allocation close to the physical interlocking when fixing routes at this granularity, which in turn supports the construction of truly conflict-free timetables.

We label the daily adjustment of a tactical timetable to actual circumstances (e.g., ad-hoc trains, construction, unexpected demand) as operational timetabling. Tactical and operational timetabling can be integrated with vehicle and crew scheduling (see Fuchs et al. (2025b) for an example of integrating vehicle scheduling). Alternatively, the tasks can be solved sequentially, which directly corresponds to the original order in the sequential process model (Liebchen and Möhring, 2007).

The DISPLIB library provides suitable instances for operational timetabling (Kloster et al., 2025). The instances provide timetables with microscopic routes, including detailed resource occupation, to choose from. Events sometimes have bounds which describe the time slots and at least the objective events for which deviation to a specified arrival time should be minimized to guide and bound the scheduling of the trains.

Although the library is called *Dispatching Library*, the information provided in the instances does not align with dispatching or disposition timetabling as recognized by the practice-inspired process model in Figure 1b. Disposition timetabling focuses on re-establishing feasibility or minimizing deviations from the planned timetable in response to disruptions or short-notice requests. It involves retiming of events, reordering of conflicting trains, and rerouting through available paths (Cacchiani et al., 2014). The difference to tactical and operational timetabling is the availability of an original microscopic timetable containing selected microscopic routes and detailed timing of events per train. Disposition timetabling can therefore be based on more available information; however, it may potentially have the same degrees of freedom in adjustment as tactical and operational timetabling.

2.2. Microscopic Modeling and Conflict Resolution

In microscopic timetabling, train movements are modeled as sequences of time-stamped operations, such as arrivals, departures, entries, and exits, each occurring at a, in constrast to macroscopic timetabling, precise, track-based physical location on the infrastructure representation. These operations are linked by activities representing runs, dwells, and headway separations. The underlying model structure is often organized as a directed graph, either derived from infrastructure topology or formalized through an Event-Activity Network (EAN) (Liebchen and Möhring, 2007; Schlechte et al., 2011). Regardless of representation, feasibility in real-life train operation depends critically on the exclusive occupation of shared infrastructure elements, such as track segments, platforms, or switches, as enforced by interlocking systems (Leutwiler and Corman, 2023). These physical constraints must be translated into resource conflicts at the model level, typically by enforcing that no two trains occupy the same resource during overlapping time intervals. In macroscopic models, resource occupation is generally modeled in a simplified manner, for example, through minimum headways or aggregated infrastructure.

Overall, microscopic timetabling formulations differ primarily along three modeling axes: the representation of time (continuous or discrete), the degree of routing flexibility (fixed or variable paths), and the encoding of mutual exclusivity (precedence, disjunction, or Boolean logic).

The representation of time can follow either a continuous or a discrete approach. Continuous-time formulations dominate the literature due to their precision and flexibility in modeling, particularly when combined with propagation-based enhancements. However, discrete-time formulations have also been proposed, typically in the form of time-expanded graphs. For instance, Caimi et al. (2011) introduce a multicommodity flow model over a discrete time-space network, where each train is modeled as a unit flow along arcs representing possible movements and track allocations. Resource conflicts are encoded as maximal cliques in a conflict graph, which arise from overlapping occupancy patterns. While this approach introduces more variables, it enables highly structured MIP formulations with strong relaxations and supports integration with safety margins or robustness buffers.

Several equivalent formulations exist to encode such resource-based separation constraints. One common approach involves introducing logical disjunctions between overlapping time intervals, ensuring that at most one train occupies a resource at any given time. Alternatively, precedence variables can be introduced to explicitly determine the order of train movements over shared sections. Both formulations can be expressed through conditional inequalities using large constants (big-M) in continuous-time models (Pascariu et al., 2022; Mannino and Nakkerud, 2023), or as Boolean satisfiability encodings in Boolean satisfiability (SAT) and Satisfiability Modulo Theories (SMT) solvers (Leutwiler and Corman, 2022; Fuchs et al., 2025a). These variants differ in solver suitability rather than modeling logic—each encodes mutual exclusivity over resources and is interchangeable in realism. The resource-based modeling is preserved between formulations using different operation modeling techniques (Cacchiani et al., 2014; Caimi et al., 2011).

Routing flexibility further distinguishes microscopic models. Some assume fixed itineraries, optimizing only over timings and ordering decisions. Others allow alternative routings over the infrastructure network, increasing expressiveness at the cost of model complexity. Notable examples include path-based MIP models (Pellegrini et al., 2015) and formulations with integrated routing and conflict resolution via valid inequalities (Pellegrini et al., 2019). Such inequalities propagate precedence decisions across mutually exclusive path pairs, and often accelerate solving by tightening the relaxation. Related techniques have been proposed for trains meeting in stations (Lamorgese et al., 2016).

These formulations can be solved using a wide range of algorithmic tools, including MIP, SAT, SMT, and dynamic programming. Recent research increasingly combines these paradigms into hybrid models, enabling scalability without compromising modeling fidelity (Borndörfer et al., 2020; Fuchs et al., 2022). Hybrid models, for example, initialize the optimization with a previously obtained initial, feasible solution from a satisfiability solver.

While models aimed at getting solved directly provide a detailed representation of the underlying operations, solving them at scale remains challenging due to the large number of potential conflicts and routing options. This motivates the integration of preprocessing, propagation, and decomposition strategies (Leutwiler and Corman, 2023). In this paper, we extend these directions by introducing new strengthening and reduction techniques that exploit topological structure, dominance among conflicts, and temporal constraints, thereby enabling scalable exact solving of large real-world timetabling instances.

2.3. Decomposition Approaches

Decomposition techniques are a natural way to address the complexity of microscopic timetabling. They rely on decoupling decisions across modular components of the problem. Leutwiler and Corman (2023) classify decomposition techniques by how and when subproblems are solved. Intuitive decomposition approaches partition the network spatially (e.g., Kroon et al. (2009)) or the timetable temporally with sliding horizons (e.g., Corman et al. (2014)). Other approaches decompose the timetabling problem variable-based (splitting routing and timing, e.g., Lamorgese and Mannino (2015)) or constraint-based (relax-and-check schemes, e.g., Leutwiler and Corman (2022)).

Recently, LBBD approaches gained attention. LBBD is a constraint-based method that separates a relaxed master model from subproblems, enforcing feasibility (Hooker and Ottosson, 2003). In timetabling, usually the master schedules train events under relaxed or partial infrastructure constraints. The subproblems detect and resolve constraint violations, often using SAT/SMT solvers for efficient feasibility checking (Leutwiler and Corman, 2022; Armando et al., 2004). Feasibility or no-good cuts are added to the master to exclude violating solutions. Aggregated or learning-based cuts improve convergence (Leutwiler et al., 2023; Varga et al., 2023).

Early work by D'Ariano et al. (2007) explored exact solution methods for microscopic train dispatching using a branch-and-bound approach based on job-shop scheduling analogies, demonstrating that combinatorial techniques can handle small-to-medium-scale networks under tight runtime constraints and laying the basis for later decomposition approaches, such as LBBD. Therein, groundbreaking work for timetabling was conducted by Lamorgese and Mannino (2015) and Lamorgese et al. (2016), who developed a geographic LBBD approach to railway rescheduling. The master problem schedules train lines on a macroscopic network where stations are nodes, while station-level subproblems operate on the microscopic infrastructure of blocks and signals. Assuming each platform has distinct inbound and outbound routes with identical running times, the station subproblem reduces to a list-coloring formulation. Infeasibilities are certified through logic proofs on interval-intersection graphs, yielding combinatorial Benders cuts. Leutwiler and Corman (2022) further extended this approach and cast the TTP directly as a disjunctive program, but retained discrete decisions in the subproblem. Their master and subproblem mirror the complete constraint structure of the original model (in contrast to the reduced microscopic subproblems in Lamorgese and Mannino (2015) and Lamorgese et al. (2016)). They introduce a new class of logic Benders cuts derived from graphical proofs of infeasibility; these cuts are expressed as disjunctions of event-precedence. To generate cuts efficiently, they apply SAT solving and further aggregate cuts to accelerate convergence. These approaches enable the solution of realistic, large-scale timetabling problems under real-time constraints by focusing computational effort on bottlenecks and deferring complex sub-decisions.

We also follow an LBBD approach similar to the framework of Leutwiler and Corman (2022), but further reduce the problem size by adding conflict constraints to the subproblem only if necessary. In addition to the advanced preprocessing, this significantly improves performance. This is combined with a direct MIP-based model; however, it minimizes conflicts first to obtain incumbent solutions. Such an approach is seldom reported in the literature (see Fuchs et al. (2025a), as an example in a slightly different context). Still, it is known to work well in industry projects, where, for example, resource occupation overlaps of a few seconds do not lead to practical infeasibility.

3. Problem Description and Formulation

The microscopic TTP that we consider concerns the selection of routes and timing decisions for a given set of trains $z \in \mathcal{Z}$ to construct a conflict-free feasible schedule. The objective is to minimize delays, typically measured as the deviation of scheduled times from desired lower bounds at critical locations or events. Importantly, solutions must be obtained within limited computational time budgets, reflecting the practical constraints of real-world planning environments.

We model the TTP using the *Event-Activity Network (EAN)* framework, a widely established approach for representing temporal scheduling constraints (Peeters, 2003). We formulate the model to capture the requirements specified by the instances in the DISPLIB benchmark library (Kloster et al., 2025). In our formulation, the network is structured as a directed graph $G = (\mathcal{E}, \mathcal{A})$ where individual trains are represented as distinct subgraph components, compactly modeling alternative routing choices.

Events $e \in \mathcal{E}$: Represent timestamps in the schedule, such as train arrivals or departures, or passing times at specific locations. Each event e is associated with:

- A timing variable $t_e \geq 0$ denoting its scheduled time
- Time bounds: $lb_e \leq t_e \leq ub_e$

Activities $a \in \mathcal{A}$: Directed edges $a = (e, e')$ which model the precedence relations and resource occupations between pairs of events of a single train, e.g., running times over track sections or dwell times at stations. Each activity a is characterized by:

- Duration constraints: $lb_a \leq t_{e'} - t_e \leq ub_a$
- An associated set of resources $R(a) \subset \mathcal{R}$. A resource $r \in R(a)$ is occupied for the time interval $[t_e, t'_e + \delta(r, a)]$ where $\delta(r, a)$ is the required reoccupation time of resource r after the end of activity a, ensuring sufficient separation between trains.

The EAN decomposes into disjoint directed acyclic subgraphs $G_z = (\mathcal{E}_z, \mathcal{A}_z)$ for each train $z \in \mathcal{Z}$, as illustrated in Figure 2a. A path from source to sink in G_z with assigned timings t_e for all events along that path represents a physical route and a schedule for train z.

The resources $r \in \mathcal{R}$ associated with the activities represent any entities that can be occupied by at most one train at a time, e.g., physical infrastructure elements such as track sections. Two activities $a, a' \in \mathcal{A}$ are *conflicting* if the following two conditions are met:

1. They share at least one resource: $R(a) \cap R(a') \neq \emptyset$
2. They belong to different trains: $a \in \mathcal{A}_z, a' \in \mathcal{A}_{z'}$ with $z \neq z'$

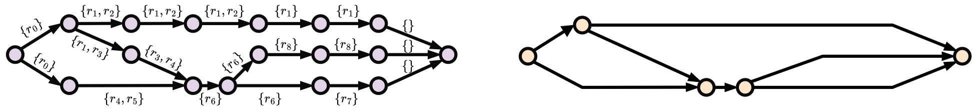
A feasible solution to the train scheduling problem must satisfy:

1. **Route Selection**: For each train $z \in \mathcal{Z}$, select exactly one path P_z from source to sink in subgraph G_z.

2. **Temporal Feasibility**: Assign a timing t_e to every event $e \in P_z$ along that path while respecting the event and activity time bounds.

3. **Resource Conflict Resolution**: Ensure any resource along the chosen path P_z is occupied by at most one train at a time.

Building on this Event-Activity Network framework, we present a logic-based optimization formulation. This formulation can be translated to either MIP or SMT solvers using standard reformulation techniques such as big-M constraints.



(a) The Event Activity Network (EAN) graph for a single train $G_z = (\mathcal{E}_z, \mathcal{A}_z)$. Events (nodes) and activities (links) with an associated set of resources $R(a)$.

(b) Train Flow Network (TFN) for the train with EAN in the Figure on the left retaining only routing decision nodes $e \in \mathcal{E}^{\mathrm{TFN}}$ and aggregating activities into edges $w \in \mathcal{W}$.

Figure 2: Example of the network representations for a train.

3.1. Route Selection

To capture routing options within the model, we employ the Train Flow Network (TFN) (Fuchs et al., 2025a), which is derived from the EAN by retaining only nodes relevant for routing decisions and aggregating the corresponding activities. In this simplified network, nodes $e \in \mathcal{E}^{\mathrm{TFN}} \subset \mathcal{E}$ represent an infrastructure decision point, such as switches or routing junctions, and each edge $w \in \mathcal{W}$ represents a possible track segment between two decision points as illustrated in Figure 2b. Thus, multiple activities from the EAN are aggregated into a single TFN edge when they form part of the same routing choice.

We introduce binary decision variables $x_w \in \{0, 1\}$ for each edge $w \in \mathcal{W}$ to indicate whether a train traverses the corresponding track segment. We use these variables to capture route dependencies in the model. Then, to ensure that a train uses exactly one route from its source node $e \in \mathcal{E}^{\mathrm{Source}}$ to its sink node $e \in \mathcal{E}^{\mathrm{Sink}}$, we introduce the following flow conservation constraints:

$$\sum_{w \in \rho^+(e)} x_w - \sum_{w \in \rho^-(e)} x_w = \begin{cases} 1 & \text{if } e \in \mathcal{E}^{\mathrm{source}} \\ -1 & \text{if } e \in \mathcal{E}^{\mathrm{sink}} \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

where $\rho^+(e)$ and $\rho^-(e)$ are the sets of outgoing and incoming edges for node e, respectively.

3.2. Temporal Feasibility

The timing constraints on events and activities depend on the route selected for each train. Consequently, these constraints must be conditionally enforced based on the binary route decision variables x_w. For every activity $a \in \mathcal{A}$, let α be the mapping which associates each activity a with the TFN edge $w = \alpha(a)$ into which it was grouped. The activity duration constraint is then activated only if the corresponding edge is selected:

$$lb_a \leq t_{e'} - t_e \leq ub_a \quad \text{if} \quad x_{\alpha(a)} = 1 \tag{2}$$

For each event $e \in \mathcal{E} \setminus \{\mathcal{E}^{\mathrm{Source}} \cup \mathcal{E}^{\mathrm{Sink}}\}$, the event is active if one of its incoming edges is used in the selected route. Thus, for the set of incoming itinerary activities $\rho^-(e)$ we have:

$$lb_e \leq t_e \leq ub_e \quad \text{if} \quad \sum_{a \in \rho^-(e)} x_{\alpha(a)} \geq 1 \tag{3}$$

3.3. Resource Conflict Resolution

Beyond routing and respecting the temporal bounds, the model must prevent resource conflicts between trains. This requires establishing a precedence order between conflicting activities. For each activity $a = (e, e') \in \mathcal{A}$, the resource occupation interval is $[t_e, t_{e'} + \delta(r, a)]$ for each resource $r \in R(a)$, where:

- t_e is the *acquire time* (when the train starts using the resource)
- $t_{e'} + \delta(r, a)$ is the *release time* (when the resource becomes available again) where $\delta(r, a)$ is the reoccupation time of resource $r \in \mathcal{R}$ after the end of activity $a \in \mathcal{A}$

For each conflicting activity pair (a_1, a_2), where $a_1 = (e_1, e'_1)$ and $a_2 = (e_2, e'_2)$ share resource r, a precedence order must hold to prevent simultaneous occupation. This requires that either train 1 completes its use of the resource before train 2 begins (i.e., $t_{e_2} \geq t_{e'_1} + \delta(r, a_1)$), or vice versa $t_{e_1} \geq t_{e'_2} + \delta(r, a_2)$. However, a precedence decision is only required if both conflicting activities are active in their respective routes. Therefore, the disjunction is conditioned on the route selection of both activities:

$$\left(t_{e_2} \geq t_{e'_1} + \delta(r, a_1) \quad \text{or} \quad t_{e_1} \geq t_{e'_2} + \delta(r, a_2)\right) \quad \text{if} \quad x_{\alpha(a_1)} = 1 \wedge x_{\alpha(a_2)} = 1 \tag{4}$$

3.4. Objective Function

Many objectives can be considered for railway scheduling problems, ranging from minimizing total travel time to fairness in delays across trains or the robustness of a schedule (Caimi et al., 2017). In this work, we adopt the objective function specified by the DISPLIB 2025 competition (Kloster et al., 2025), which minimizes delays as follows.

The objective is to minimize delay costs across a selected subset of critical events $O \subset \mathcal{E}$. For each event $e \in O$, we define a delay threshold $\bar{t}_e$ after which delay penalties accumulate. The delay cost is composed of two components:

- A linear delay term $c_e \cdot \max\{0, t_e - \bar{t}_e\}$, where c_e is a given linear delay penalty constant.
- A fixed term given by penalty d_e incurred whenever a delay occurs, modelled by the Heaviside step function $H(t_e - \bar{t}_e)$.

Together, the total objective is then given by:

$$\min \sum_{e \in O} (c_e \cdot \max\{0, t_e - \bar{t}_e\} + d_e \cdot H(t_e - \bar{t}_e)) \tag{5}$$

4. Model Strengthening and Reduction

To cope with the complexity induced by the scale and detail of microscopic TTP instances, we apply a series of preprocessing and model-strengthening steps that shrink the model, tighten the relaxations, and streamline conflict handling. We follow two complementary strategies: (a) reduce the number of variables and constraints via structural simplifications, and (b) strengthen the model with implied bounds and logic that eliminate infeasible or redundant regions of the search space *a priori*[1]. Concretely, we apply five transformations before solving, in the dependency order shown in Figure 3:

[1]The modeling techniques termed *traversal bounds* and *chunks* were developed in collaboration with Swiss Federal Railways (SBB), where similar concepts have also proven effective in practice.

1. **Time-window tightening via bound propagation**, which updates $[lb_e, ub_e]$ on events using minimal activity durations in a forward/backward pass.

2. **Topological clustering of routing nodes**, which groups parallel events into clusters with a shared time variable t_C, preserving feasibility across routing alternatives.

3. **Traversal bounds**, which impose always-active lower bounds between structural waypoints (consecutive maximal clusters or routing separators), improving propagation and LP strength independent of fractional route choices.

4. **Aggregation into train-specific resource chunks**, which collapses contiguous (possibly internally parallel) same-resource usages into single acquire/release intervals, reducing the number of precedence disjunctions.

5. **Sharing precedence for linked conflicts**, which identifies pairs of chunk-level conflicts whose relative train order is structurally fixed on both itineraries and ties or eliminates redundant disjunctions so that a single (representative) precedence decision suffices for all linked pairs.

Each step preserves the feasibility and optimality of the original problem.

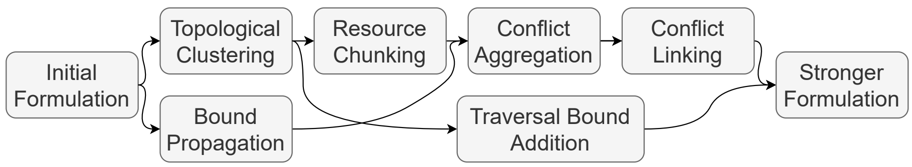


Figure 3: Order in which preprocessing and strengthening steps must be applied.

4.1. Bound Tightening via Propagation

We propagate time bounds through the event-activity network to tighten the bounds of events, considering routing alternatives, but temporarily ignoring infrastructure conflicts. The time variable t_e for each event $e \in \mathcal{E}$ is constrained within $[lb_e, ub_e]$. Bound propagation updates and tightens these bounds by considering the temporal dependencies in the EAN: each event's time is constrained by the earliest possible arrival from any preceding event (forward propagation, i.e., tightening the lower bounds) and by the latest possible departure to any succeeding event (backward propagation, i.e., tightening the upper bounds).

Forward Pass. We tighten lower bounds by considering the minimum time required to reach each event from any of its predecessors:

$$lb_e \leftarrow \max\left(lb_e, \min_{a=(e',e)\in\rho^-(e)} (lb_{e'} + lb_a)\right)$$

Backward Pass. We tighten upper bounds by considering the latest time allowed to depart from each event to any of its successors:

$$ub_e \leftarrow \min\left(ub_e, \max_{a=(e,e')\in\rho^+(e)} (ub_{e'} - lb_a)\right)$$

The bounds are updated in topological order, first forward and then backward, in a single pass over the graph. Thus, the procedure is linear in the size of the network, $O(|\mathcal{E}| + |\mathcal{A}|)$, and monotonic.

We use only the minimal durations lb_a for propagation, which are always defined. Bound propagation aims to accelerate solving time by reducing the search space while guaranteeing feasibility and optimality. Additionally, to heuristically strengthen the problem, especially in the absence of explicit upper bounds, one can impose a conservative global delay cap before performing propagation. However, this global delay must be chosen carefully, as otherwise, one might over-restrict the instance.

4.2. Clustering of Topologically Parallel Nodes

To reduce the number of time variables t_e in the event-activity network, we cluster topologically parallel events of the same train. The motivation is that even if a train has multiple alternative routes available, exactly one route will be selected. Consequently, at most one event per set of topologically parallel events will ever be active. Rather than modeling all individual t_e variables for each parallel event and enforcing a disjunction, we aim to group the parallel events in a cluster with a shared time variable t_C. This eliminates inactive variables and reduces the number of conditional constraints.

Our clustering algorithm assigns clusters as follows: recall from Section 3 that the EAN decomposes into disjoint directed acyclic subgraphs $G_z = (\mathcal{E}_z, \mathcal{A}_z)$ for each train $z \in \mathcal{Z}$. Within each subgraph G_z, we sort the events topologically, assigning each event a rank. We start with the lowest rank, at the source event. We can add an event to a cluster if (1) it has a higher rank and (2) is not a successor (reachable) of any of the events already present in the cluster. Thus, on any route chosen for train z, only a single event from each cluster is passed. We acknowledge the non-uniqueness of clusterings and this procedure produces one possible clustering greedily. Figure 4 illustrates the effect of this clustering procedure for one train.

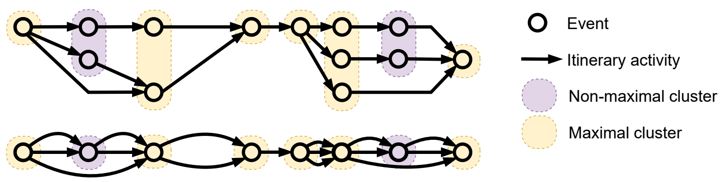


Figure 4: Clustering of topologically parallel events into maximal (yellow) and non-maximal (purple) clusters. Each cluster group has events at the same topological rank.

A cluster is *maximal* if all train routes pass through at least one of its nodes. Otherwise, it is *non-maximal* and may be bypassed depending on routing decisions. Each cluster $C \subset \mathcal{E}_z$ is assigned a shared time variable t_C. All participating events are substituted: $t_e \leftarrow t_C$ for all $e \in C$. The time bounds of the cluster are set to cover all alternatives:

$$lb_C := \min_{e \in C} lb_e, \qquad ub_C := \max_{e \in C} ub_e$$

These bounds are safe for all routes. If the selected route activates a specific $e \in C$ with tighter bounds, these are then enforced conditionally.

This substitution preserves feasibility and optimality, while reducing the number of time variables and improving the compactness of the formulation.

In Figure 4, maximal and non-maximal clusters are shown in different colors. The resulting clustered EAN serves as the foundation for constructing strengthening traversal bounds between clusters in the next step.

4.3. Improving Minimal Duration Propagation with Traversal Bounds

Traversal bounds are additional timing constraints that impose minimal durations between structural waypoints in a train's itinerary. They enforce lower bounds on the separation of consecutive maximal clusters, which must always be traversed by a train in a fixed order. Traversal bounds do not restrict feasibility; they are implied by the subpaths connecting structural waypoints, and thus, they can be omitted without affecting correctness. Their purpose is to strengthen the formulation by including implied information.

We define the set of all traversal bound constraints as $\mathcal{B} \subseteq \mathcal{E} \times \mathcal{E}$, where each pair $(e, e') \in \mathcal{B}$ represents a minimal duration constraint between two structural events that are guaranteed to be traversed by the train. For each such pair, we enforce a lower-bound constraint of the form:

$$t_{e'} - t_e \geq d^{\min}_{e,e'} \qquad \forall (e, e') \in \mathcal{B}$$

where $d^{\min}_{e,e'}$ is the minimal travel time between e and e' over any valid route in the train's itinerary, computed via shortest-path propagation using minimal activity durations.

We define two types of traversal bounds, both derived from the EAN and computed before solving: *cluster traversal bounds* connecting consecutive maximal clusters in the event-activity network, and *routing traversal bounds* spanning routing regions identified in the train flow network.

4.3.1. Cluster Traversal Bounds

After clustering topologically parallel events (see Section 4.2), each train has a sequence of maximal clusters $(C_1, C_2, \ldots, C_k)$. These clusters are guaranteed to be visited in the given order along any feasible route. For each consecutive pair (C_i, C_{i+1}), we define a traversal bound with implied duration

$$t_{C_{i+1}} - t_{C_i} \geq d^{\min}_{i,i+1}$$

where $d^{\min}_{i,i+1}$ is the shortest-path duration over the EAN from any event in C_i to any event in C_{i+1}, using minimal activity durations. The variables t_{C_i} are the shared time variables introduced during the clustering process.

4.3.2. Routing Traversal Bounds

Independent of clustering, we detect routing regions in the train flow network. A routing traversal bound starts at a *divergence node* (where multiple routing paths depart) and ends at the corresponding *convergence node* (where all alternatives rejoin). All feasible routes pass these nodes, thus corresponding to vertex separators. Let $(e, e') \in \mathcal{B}$ denote the associated events marking the entry and exit of the routing region. The traversal bound constraint

$$t_{e'} - t_e \geq d^{\min}_{e,e'}$$

is then added to the set $\mathcal{B}$, where $d^{\min}_{e,e'}$ is the shortest duration over all routing alternatives within the traversal bound.

A comparable idea for routing traversal bounds was introduced by Samà et al. (2016), who used *fictitious operations* in a relaxed alternative graph to represent the shortest path between two real operations across routing alternatives, assigning each a fixed minimal traversal time. In contrast, our traversal bounds incorporate this aggregation permanently as explicit timing constraints between structural separators, rather than as temporary constructs in a relaxed model.

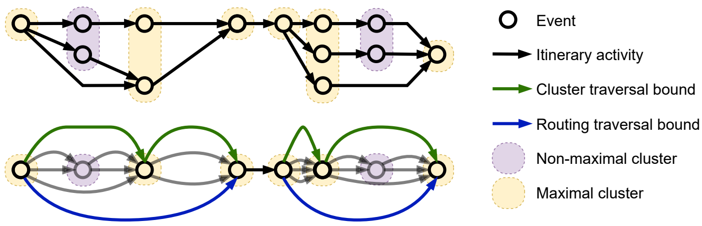


Figure 5: The EAN of an example train, showing the two types of traversal bounds: cluster traversal bounds (green) enforce minimal travel durations between consecutive maximal clusters, while route traversal bounds (blue) span routing regions between areas with parallel routing options.

Both types of traversal bounds are illustrated in Figure 5: cluster traversal bounds appear between the colored clusters in the top part, while routing traversal bounds are shown in the lower part between routing separators. These constraints are added statically before optimization and require no additional variables or binary activation logic.

4.3.3. *Why Traversal Bounds strengthen the Linear Relaxation*

To illustrate the benefits of traversal bounds, we present a minimal example that demonstrates how they eliminate a structural weakness in the LP relaxation. For simplicity, we consider two consecutive events $e, e' \in \mathcal{E}_z$ connected by parallel activities; the same reasoning can be generalized to cluster variables t_C between maximal clusters and routing regions.

Consider the two events with time variables t_e and $t_{e'}$ connected by two parallel activities $a_1, a_2 \in \mathcal{A}_z$ corresponding to alternative routing paths. Suppose both activities have identical minimal durations:

$$lb_{a_1} = lb_{a_2} = 1,$$

Let x_{w_1}, x_{w_2} denote the binary routing variables for the corresponding TFN edges, satisfying $x_{w_1} + x_{w_2} = 1$. The standard linearization of the conditional timing constraints (Equation (2)) enforces:

$$t_{e'} - t_e \ \geq \ lb_{a_1}\, x_{w_1}, \qquad t_{e'} - t_e \ \geq \ lb_{a_2}\, x_{w_2}.$$

In the LP relaxation, where $0 \leq x_{w_1}, x_{w_2} \leq 1$ and $x_{w_1} + x_{w_2} = 1$, a fractional solution $x_{w_1} = x_{w_2} = \frac{1}{2}$ satisfies both constraints with

$$t_{e'} - t_e \ \geq \ \max\{lb_{a_1} x_{w_1},\, lb_{a_2} x_{w_2}\} \ = \ \tfrac{1}{2},$$

Thus, the LP relaxation permits e and e' to be separated by only $\frac{1}{2}$ time units, despite every feasible integer solution requiring a full unit.

Adding a route-independent traversal bound constraint

$$t_{e'} - t_e \ \geq \ d^{\min}_{e,e'}, \qquad d^{\min}_{e,e'} \ = \ \min\{lb_{a_1}, lb_{a_2}\} \ = \ 1,$$

restores the correct separation for any routing variables (x_{w_1}, x_{w_2}), including fractional ones. This tightening is particularly valuable when t_e is shifted above its lower bound: without the traversal bound, the LP may set $t_{e'} = t_e + \frac{1}{2}$, whereas the traversal bound enforces $t_{e'} \geq t_e + 1$, propagating timing consistently throughout the network.

The effect scales with the number k of parallel alternatives. With k parallel activities and routing variables satisfying $\sum_{r=1}^{k} x_r = 1$, a symmetric fractional solution $x_r = \frac{1}{k}$ yields $t_j - t_i \geq \max_r\{lb_{a_r} x_r\} = \frac{1}{k} \cdot \max_r lb_{a_r}$, which can be much smaller than $\min_r lb_{a_r}$. A single traversal bound $t_j - t_i \geq \min_r lb_{a_r}$ eliminates this relaxation gap.

4.4. Aggregation of Resources into Chunks

To reduce the number of precedence decisions and obtain a more compact formulation, we aggregate consecutive and parallel activities sharing the same resource into *resource chunks* (or simply *chunks*).

Each chunk $q \in Q$ represents an aggregation of one or more paths along a single train's network during which the same physical resource is occupied. A chunk is a (not necessarily connected) subgraph of a train's EAN $G_z = (\mathcal{E}_z, \mathcal{A}_z)$ comprising a set of activities $\mathcal{A}_q \subset \mathcal{A}_z$ that satisfy the following conditions:

- All activities in the chunk use the same resource: $\exists r \in \mathcal{R}$ such that $r \in R(a)$ for all $a \in \mathcal{A}_q$.
- All activities have the same reoccupation time requirement: $\delta(r, a) = \delta(r, q)$ for all $a \in \mathcal{A}_q$, where $\delta(r, q)$ denotes the reoccupation time of the chunk.
- The resource is acquired once and released once (i.e., no internal re-entries or multi-use within the chunk).

By construction, each train traverses any given chunk at most once. However, a chunk may be bypassed by alternative routes. Chunks may contain parallel routes of the same train (i.e., alternative routing through the chunk). Otherwise, the chunks consist of connected activities. Figure 6 illustrates examples of valid and invalid configurations.

These conditions ensure that the chunk can be represented by a single acquisition–release time interval and a single pairwise disjunction for conflict resolution. Let $\mathcal{W}_q^{\text{in}}, \mathcal{W}_q^{\text{out}} \subset \mathcal{W}$ denote the sets of TFN edges that enter and exit the chunk subgraph. Each $w \in \mathcal{W}_q^{\text{in}}$ is mapped to an entry event $e^{\text{in}}(w) \in \mathcal{E}_z$, and each $w' \in \mathcal{W}_q^{\text{out}}$ is mapped to an exit event $e^{\text{out}}(w') \in \mathcal{E}_z$ on the boundary of the chunk subgraph. We introduce two auxiliary time variables per chunk: the acquisition time τ_q^{acq} and release time τ_q^{rel}. Their domains are set to cover the route-specific bounds:

$$\tau_q^{\text{acq}} \in \left[\min_{w \in \mathcal{W}_q^{\text{in}}} lb_{e^{\text{in}}(w)},\ \max_{w \in \mathcal{W}_q^{\text{in}}} ub_{e^{\text{in}}(w)}\right], \quad \tau_q^{\text{rel}} \in \left[\min_{w' \in \mathcal{W}_q^{\text{out}}} lb_{e^{\text{out}}(w')},\ \max_{w' \in \mathcal{W}_q^{\text{out}}} ub_{e^{\text{out}}(w')}\right].$$

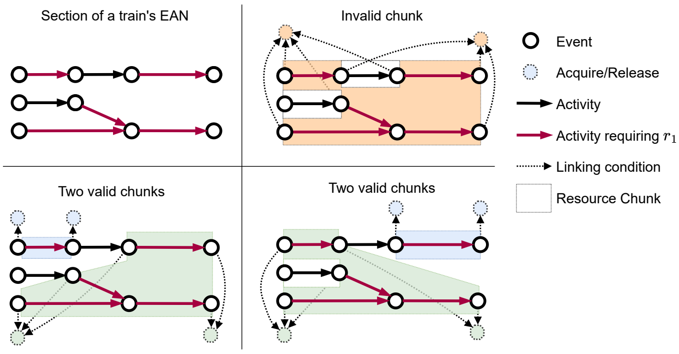


Figure 6: Illustration of valid and invalid chunk configurations. The top left is an example of a section of a train's EAN G_z, where some activities require a resource r_1 (red arcs). The top right shows an invalid chunk with internal re-entry of the same resource, while the two bottom examples illustrate two valid ways to combine the activities in chunks. The concrete implementation considers combining parallel chunks based on the earliest timing, resulting in the chunk on the lower right.

We need to ensure that the chunk acquisition and release times are set according to the selected route. Because chunks may contain parallel routing paths, multiple entry or exit points can be simultaneously active depending on which route is chosen. The acquisition and release times must therefore capture the full resource occupation interval: the earliest entry point and the latest exit point among all active routing alternatives within the chunk. Thus, for each TFN edge that is traversed ($x_w = 1$), its corresponding entry or exit event-time bounds the acquisition or release time:

$$\forall w \in \mathcal{W}_q^{\text{in}} : \quad x_w = 1 \Rightarrow \tau_q^{\text{acq}} \leq t_{e^{\text{in}}(w)}, \qquad \forall w' \in \mathcal{W}_q^{\text{out}} : \quad x_{w'} = 1 \Rightarrow \tau_q^{\text{rel}} \geq t_{e^{\text{out}}(w')}.$$

If the chunk is not traversed, no TFN edge from $\mathcal{W}_q^{\text{in}} \cup \mathcal{W}_q^{\text{out}}$ is active, and τ_q^{acq} and τ_q^{rel} remain free within their domains. Also, if the acquisition or release coincides with a previously identified maximal cluster (Section 4.2), the auxiliary variable is replaced by the corresponding cluster time variable.

4.4.1. Conflict Detection and Precedence over Chunks

To prevent simultaneous use of a shared infrastructure element, we define disjunctive precedence constraints between conflicting chunks. Two chunks q and q' from different trains are considered in conflict if they use the same resource r and their tightened time windows (Section 4.1) do not prohibit temporal overlap.

For activation, we only need to watch *either* the entry or the exit alternatives, since a valid chunk is traversed at most once. Let

$$\mathcal{W}_q := \begin{cases} \mathcal{W}_q^{\text{in}} & \text{if } |\mathcal{W}_q^{\text{in}}| \leq |\mathcal{W}_q^{\text{out}}| \\ \mathcal{W}_q^{\text{out}} & \text{otherwise} \end{cases} \qquad x_q := \bigvee_{w \in \mathcal{W}_q} (x_w = 1)$$

and analogously $\mathcal{W}_{q'}$ and $x_{q'}$. Here, x_q indicates whether chunk q is active (i.e., at least one of its entry or exit edges is selected). The conflict between chunks q and q' sharing resource r is then modeled as a guarded disjunction:

$$(x_q \wedge x_{q'}) \Rightarrow \left(\tau_{q'}^{\text{acq}} - \tau_q^{\text{rel}} \geq \delta(r, q) \quad \vee \quad \tau_q^{\text{acq}} - \tau_{q'}^{\text{rel}} \geq \delta(r, q')\right). \tag{6}$$

This ensures that, if both chunks are on the selected routes, one must follow the other, respecting the reoccupation time. For chunks that are always active (e.g., on routing cuts or with unique alternatives), we can omit the activation, and Equation (6) reduces to an unconditional precedence disjunction. The disjunctive form is solver-agnostic; linearizations such as indicator or big-M constraints may be introduced later for MIP solving.

4.5. Representative Precedence for Linked Conflicts

In some situations, multiple chunk-level conflicts between two trains encode the same underlying precedence decision. This occurs when the *EAN* of each train enforces a consistent relative order across multiple pairs of conflicting chunks using the same shared resource. We refer to such conflicts as *linked* conflicts.

Formally, consider two conflicting chunk pairs (q_1, q'_1) and (q_2, q'_2), where q_i and q'_i are chunks of trains z and z' respectively. The pairs are *linked* if two conditions are met: (i) the itinerary of z enforces a direct order between q_1 and q_2, written as $q_1 \prec q_2$, and (ii) the itinerary of z' also enforces a fixed order between q'_1 and q'_2 as $q'_1 \prec q'_2$. An enforced direct order means that all entry nodes of q_2 or q'_2 are directly reachable from some exit node of q_1 or q'_1, respectively, including the possibility that the chunks overlap. In this case, the precedence decision for (q_1, q'_1) necessarily implies that for (q_2, q'_2), and vice versa.

Thus, if all chunks are active, we can link the two precedence decisions with the following constraint:

$$(x_{q_1} \wedge x_{q'_1} \wedge x_{q_2} \wedge x_{q'_2}) \Rightarrow \begin{cases} \quad \tau^{\text{acq}}_{q'_1} - \tau^{\text{rel}}_{q_1} \geq \delta(r, q_1) \quad \wedge \quad \tau^{\text{acq}}_{q'_2} - \tau^{\text{rel}}_{q_2} \geq \delta(r, q_2) \\ \vee \quad \tau^{\text{acq}}_{q_1} - \tau^{\text{rel}}_{q'_1} \geq \delta(r, q'_1) \quad \wedge \quad \tau^{\text{acq}}_{q_2} - \tau^{\text{rel}}_{q'_2} \geq \delta(r, q'_2) \end{cases}$$

This constraint prevents the solver from assigning opposite precedence directions to successive chunks whose relative order is structurally fixed. It can be used either to retain both pairs with a logical tie or to eliminate one pair and propagate the chosen decision from the remaining pair to the other.

Our approach extends the precedence simplification strategy introduced in the RECIFE-MILP model by Pellegrini et al. (2019), where conflicts across multiple track-circuits are aggregated into a single representative precedence variable when the relative train order is structurally enforced. While their model operates at the level of track elements, our method extends this reasoning to larger EAN-based resource chunks with explicit acquisition and release times. An illustrative example of two linked chunk pairs and two unlinked pairs between two trains is shown in Figure 7.

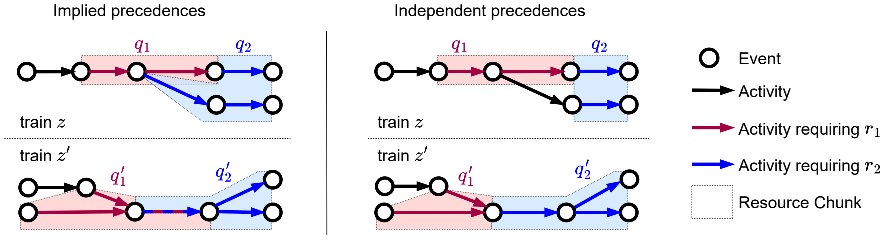


Figure 7: On the left and right, examples of two successive resource chunks (shaded red and blue) between trains z (top) and z' (bottom) that make up conflict pairs (q_1, q'_1) and (q_2, q'_2). The left half shows $q_1 \prec q_2$ and $q'_1 \prec q'_2$ leading to an implied precedence. In the right half, no such implication holds as $q_1 \not\prec q_2$ even though $q'_1 \prec q'_2$. Note, an overlap between chunks may also imply direct reachability $q'_1 \prec q'_2$, as in the lower left (red and blue dashed arrow).

5. Compact Direct Model with Disjunctions

We now summarize the complete direct model as a compact MIP model. This model integrates all enhancements from the previous sections, including route-dependent timing, resource chunk aggregation, structural clustering of events, and traversal bounds. All conditional constraints are expressed using logic-based implications or disjunctive guards, allowing flexible translation to MIP or SMT solvers. All variables, sets, and functions used in the formulation are summarized in Table 1.

Table 1: Model notation used in the compact MIP formulation.

Symbol	Description
$\mathcal{Z}$	Set of trains
$\mathcal{E}$	Event set (includes cluster representatives t_C and unclustered t_e)
$\mathcal{C}$	Set of clusters; $C \in \mathcal{C}$ is a set of events
$\mathcal{A}$	Activities (e, e') with durations $[lb_a, ub_a]$
$\mathcal{E}^{\mathrm{TFN}} \subset \mathcal{E}$	Set of events in the TFN.
$\mathcal{W}$	TFN edges with binary routing variables x_w
$\alpha : \mathcal{A} \to \mathcal{W}$	Map assigning each activity to its TFN edge
$\rho^+, \rho^- : \mathcal{E}^{\mathrm{TFN}} \to \mathcal{W}$	Outgoing/incoming TFN edges at event e
$\mathcal{B}$	Traversal-bound pairs $(u, v) \in \mathcal{E} \times \mathcal{E}$ with $d_{uv}^{\min}$
$\mathcal{Q}$	Set of resource chunks (per train) with activation variable x_q
$\mathcal{R}$	Set of resources
$r : \mathcal{Q} \to \mathcal{R}$	$r(q)$ is the resource used by chunk $q \in \mathcal{Q}$
$\mathcal{W}_q^{\mathrm{in}}, \mathcal{W}_q^{\mathrm{out}} \subset \mathcal{W}$	TFN entry/exit edges for chunk $q \in \mathcal{Q}$
$\mathcal{W}_q \subset \mathcal{W}$	For each $q \in \mathcal{Q}$ the smaller set from $\{\mathcal{W}_q^{\mathrm{in}}, \mathcal{W}_q^{\mathrm{out}}\}$
$e^{\mathrm{in}}(w), e^{\mathrm{out}}(w) \subset \mathcal{E}$	Events or clusters associated with chunk entry/exit edge w
$\delta : \mathcal{R} \times \mathcal{A} \to \mathbb{R}_{\geq 0}$	$\delta(r, a)$ is the reoccupation time of resource $r \in \mathcal{R}$ after end of activity $a \in \mathcal{A}$.
$d_{ee'}^{\min}$	Minimum travel time between e and e' (for traversal bounds)
$\mathcal{O} \subset \mathcal{E}$	Delay-relevant events with thresholds $\bar{t}_e$, costs c_e, d_e
$t_e \in \mathbb{R}^+$	Time of event $e \in \mathcal{E}$
$x_w \in \mathbb{B}$	Binary variable that is 1 if the track segment corresponding to edge $w \in \mathcal{W}$ is traversed by a train and 0 otherwise
$x_q \in \mathbb{B}$	Binary variable that is 1 if the chunk $q \in \mathcal{Q}$ is active and 0 otherwise.
$\tau_q^{\mathrm{acq}},\ \tau_q^{\mathrm{rel}} \in \mathbb{R}^+$	Acquisition and release times of chunk $q \in \mathcal{Q}$, respectively

Objective function:

$$\min \quad \sum_{e \in \mathcal{O}} \left(c_e \cdot \max\{0, t_e - \bar{t}_e\} + d_e \cdot H(t_e - \bar{t}_e)\right) \tag{M0}$$

Route selection constraints:

$$\sum_{w \in \rho^+(e)} x_w - \sum_{w \in \rho^-(e)} x_w = \begin{cases} 1 & e \in \mathcal{E}_z^{\mathrm{source}} \\ -1 & e \in \mathcal{E}_z^{\mathrm{sink}} \\ 0 & \text{otherwise} \end{cases} \qquad \forall e \in \mathcal{E}^{\mathrm{TFN}} \tag{M1}$$

Conditional timing constraints:

$$x_w = 1 \Rightarrow lb_a \leq t_{e'} - t_e \leq ub_a \qquad \forall a = (e, e') \in A,\ \alpha(a) = w \tag{M2}$$

Always-active traversal bounds:

$$t_{e'} - t_e \geq d_{ee'}^{\min} \qquad \forall (e, e') \in \mathcal{B} \tag{M3}$$

Chunk activation and timing:

$$x_w = 1 \Rightarrow \tau_q^{\text{acq}} \le t_{e^{\text{in}}(w)} \qquad \forall q \in \mathcal{Q},\ w \in W_q^{\text{in}} \tag{M4a}$$

$$x_w = 1 \Rightarrow \tau_q^{\text{rel}} \ge t_{e^{\text{out}}(w)} \qquad \forall q \in \mathcal{Q},\ w \in W_q^{\text{out}} \tag{M4b}$$

Conflict resolution over resource chunks:

$$x_q := \bigvee_{w \in \mathcal{W}_q} (x_w = 1) \qquad \forall q \in \mathcal{Q} \tag{M5a}$$

$$(x_q \wedge x_{q'}) \Rightarrow \left(\tau_{q'}^{\text{acq}} - \tau_q^{\text{rel}} \ge \delta(r(q), q)\ \vee\ \tau_q^{\text{acq}} - \tau_{q'}^{\text{rel}} \ge \delta(r(q'), q')\right) \qquad \forall (q, q') \text{ in conflict} \tag{M5b}$$

Cluster substitution constraints:

$$\bigvee_{w \in \rho^-(e)} (x_w = 1) \Rightarrow t_e = t_C \qquad \forall C \in \mathcal{C},\ \forall e \in C \tag{M6a}$$

$$\bigvee_{w \in \rho^-(e)} (x_w = 1) \Rightarrow lb_e \le t_C \le ub_e \qquad \forall C \in \mathcal{C},\ \forall e \in C \tag{M6b}$$

Variable domains:

$$t_e \in [lb_e, ub_e] \qquad \forall e \in \mathcal{E} \tag{M7a}$$

$$x_w \in \{0, 1\} \qquad \forall w \in \mathcal{W} \tag{M7b}$$

$$x_q \in \{0, 1\} \qquad \forall q \in \mathcal{Q} \tag{M7c}$$

$$\tau_q^{\text{acq}} \in \left[\min_{w \in \mathcal{W}_q^{\text{in}}} lb_{e^{\text{in}}(w)},\ \max_{w \in \mathcal{W}_q^{\text{in}}} ub_{e^{\text{in}}(w)}\right] \qquad \forall q \in \mathcal{Q} \tag{M7d}$$

$$\tau_q^{\text{rel}} \in \left[\min_{w \in \mathcal{W}_q^{\text{out}}} lb_{e^{\text{out}}(w)},\ \max_{w \in \mathcal{W}_q^{\text{out}}} ub_{e^{\text{out}}(w)}\right] \qquad \forall q \in \mathcal{Q} \tag{M7e}$$

The objective function (M0) minimizes total delay using the DISPLIB-specific linear and fixed penalties. Flow conservation over the train flow network is enforced in Equation (M1). The conditional activity duration constraints (M2) are active only when the corresponding TFN edge is selected. Always-active traversal bounds are added in Equation (M3) between structural separators such as maximal clusters or routing separators. Resource-chunk entry and exit times are linked to their route-dependent access points in Equations (M4a)–(M4b), and conflicting chunks are mutually separated via disjunctive precedence in Equation (M5b), guarded by activation literals from Equation (M5a). Equations (M6a)–(M6b) implement clustering: the cluster time variable t_C substitutes member event times when the route selects $e \in C$, and only the bounds of activated events are enforced on t_C, which keeps the cluster bounds safe across all routes. Finally, Equations (M7a)–(M7e) define the domains of all decision variables.

6. Solution Methods

To solve the reformulated model introduced in Section 5, we employ two complementary exact optimization techniques: a *Soft-Conflict MIP* (`SC-MIP`) method and a *Conflict-Discovery*

Logic-Based Benders Decomposition (`CD-LBBD`) approach. Each method exploits different structural features of the problem. Relaxing conflict resolution to soft constraints aims at guiding the solver to feasible solutions quickly, yielding warm starts for subsequent improvement. `CD-LBBD`, on the contrary, leverages the separability of coordination and feasibility decisions. Both methods are executed in parallel and contribute either feasible solutions or dual bounds. Section 6.1 presents the `SC-MIP` method and Section 6.2 describes the `CD-LBBD` approach.

6.1. Two-Stage MIP Approach via Soft Conflict Relaxation (`SC-MIP`)

We derive a MIP formulation of the model in Section 5 by linearizing the disjunctive precedence constraints over resource chunks using standard big-M techniques. The linearized model can be found in Appendix A. Binary selection variables encode each disjunction, and the associated big-M values are tightened based on propagated event time bounds, following the approach of Pellegrini et al. (2015). Solving the resulting model directly with hard precedence disjunctions, however, becomes computationally challenging due to the large number of potential conflicts and binary decisions. To mitigate this, we adopt a two-stage solution strategy: we first compute a conflict-free schedule using a soft-conflict model where the conflict resolution constraints are relaxed, and subsequently solve the original hard-disjunction model using the conflict-free solution as a warm start.

In the first stage, we replace each disjunctive constraint over conflicting chunk pairs (q, q') with relaxed inequalities that permit overlapping chunks but penalize them. As described in the linearized model in Appendix A, each pair is associated with a binary variables $y_{qq'}, y_{q'q} \in \{0, 1\}$ to select the precedence direction $q \prec q'$ or $q' \prec q$, respectively. For the relaxation, we introduce a continuous nonnegative violation variable $v_{qq'} \geq 0$ to capture any residual conflict in overlapping chunks. The objective of this relaxed model is to minimize the total amount of conflict violation:

$$\min \sum_{(q,q')} v_{qq'} \tag{SC-o}$$

The relaxed precedence constraints replace the precedence constraints of the linearized model in Equations (LM5b-qq') and (LM5b-$q'q$) as:

$$\tau_{q'}^{\text{acq}} - \tau_{q}^{\text{rel}} \geq \delta_{qq'} - M_{qq'}^{\text{conf}} \cdot (1 - y_{qq'}) - v_{qq'} \tag{SC-qq'}$$

$$\tau_{q}^{\text{acq}} - \tau_{q'}^{\text{rel}} \geq \delta_{q'q} - M_{q'q}^{\text{conf}} \cdot (1 - y_{q'q}) - v_{qq'} \tag{SC-$q'q$}$$

To compute a conflict-free schedule, we embed the relaxed MIP model into the following iterative refinement loop:

1. *Initialization:* Each train is assigned its shortest path in the EAN, ignoring conflicts, which fixes all sets of event times t_e to the lower bounds lb_e.
2. *Conflict minimization:* The relaxed model is solved to minimize conflict violations.
3. *Time window update:* If a conflict-free solution is found, proceed to Step 4. Otherwise, we update the solution space as follows and return to Step 2.
    - For the trains involved in conflicts, we relax their routing options and update event time windows to $[t_e - \hat{t}/2, t_e + \hat{t}]$, where $\hat{t}$ is a predefined parameter (approximately the maximal expected delay, e.g., 1 hour).

- For trains unaffected by conflicts, we fix the routing and restrict the event time windows to $[t_e - \hat{t}/6, t_e + \hat{t}/6]$.

  The adjusted time bounds are propagated through the resulting event-activity network as described in Section 4.1.

4. *Termination:* The conflict-free solution is passed as a warm-start to the original model with hard precedence constraints reintroduced and the original routing options and time bounds restored.

Once a conflict-free schedule is obtained ($v_{qq'} = 0$ for all pairs), the soft-conflict model is transferred to the full formulation with the original linearized hard precedence constraints from Equations (LM5b-qq') and (LM5b-$q'q$).

The objective is switched from conflict minimization to the original delay cost function defined in Equation (M0). The event time windows are reset to their original bounds, and the model is then solved to optimality within the given time limit.

The iterative refinement loop is designed to steer the solver toward feasible regions without requiring all precedence disjunctions to be modeled upfront. However, in some cases, the loop can converge prematurely to a structurally infeasible configuration. For instance, a pair of trains with a violated conflict may be blocked by a third train that is not directly implicated in any conflict and therefore remains fixed. Such local entanglements can prevent convergence unless all structurally relevant trains are simultaneously relaxed.

The method is particularly intended for instances with high routing flexibility and large numbers of alternative routes, where we expect the likelihood of becoming stuck in local infeasibilities to be low. The combination of routing diversity and overlapping time windows increases the potential for successful rerouting and timing adjustments during conflict minimization. These structural characteristics motivated the two-stage procedure, which is specifically designed to exploit temporal and routing flexibility in microscopic timetabling models.

6.2. *Logic-Based Benders Decomposition with Conflict-Discovery (`CD-LBBD`)*

We apply LBBD to exploit structural separation in the TTP problem. While the complete model (eq. (M0) - eq. (M7e)) includes routing, timing, and conflict resolution for all trains, only a small subset of events, $\mathcal{O} \subset \mathcal{E}$, affects the objective. Furthermore, many microscopic conflicts are either irrelevant (i.e., due to routing or timing) or can be resolved locally once high-level coordination is established. This motivates a decomposition where the coordination of critical events is handled in a compact master problem, and microscopic feasibility is enforced in a separate subproblem.

In our LBBD framework, the master problem controls only the timing of a selected subset of events, typically those that contribute to the objective function (i.e., the set $\mathcal{O}$). It proposes candidate values for these so-called coordination events and minimizes the objective value. All routing decisions, detailed timing, and conflict resolutions are delegated to the subproblem, which checks whether a microscopically feasible schedule exists that respects the fixed master times. This separation enables us to reduce the size of the master model.

The subproblem implements the complete microscopic model as described in Section 5, including activity durations, traversal bounds, and disjunctive conflict constraints over resource chunks. However, conflict constraints are not encoded upfront. Instead, we maintain a dynamic conflict set, which is initially empty. Upon solving the subproblem for a given master solution, we scan the resulting schedule for any violated and not yet included conflicts. If any are found, they are added to the conflict set, and the subproblem is re-solved. This lazy discovery loop continues until no further violations occur. If the subproblem is infeasible, a certificate is extracted and projected back to the master via a disjunctive Benders cut.

Each infeasibility certificate derived by the subproblem corresponds to a set of disjunctive timing constraints. At least one of them must hold to avoid the conflict in future iterations. The cut is added to the master using indicator variables to enforce the disjunction:

$$\bigvee_{k=1}^{m}\left(t_{e_b^k} - t_{e_a^k} \geq \underline{\Delta}_k\right) \quad \Longleftrightarrow \quad \sum_{k=1}^{m} u_k \geq 1, \quad u_k = 1 \Rightarrow t_{e_b^k} - t_{e_a^k} \geq \underline{\Delta}_k$$

where each u_k is a binary selector variable.

To avoid redundancies, we apply dominance filtering. A cut is retained only if it is not implied by an existing one with weaker or equivalent separation logic over the identical event pairs. This ensures that the master problem remains compact even after multiple iterations.

The architecture is illustrated in Figure 8. The master MIP proposes candidate coordination times. The subproblem, implemented using an SMT-based difference logic solver based on Leutwiler and Corman (2022), verifies feasibility under routing and infrastructure constraints. If the subproblem is infeasible, a logic-based Benders cut excludes the current master proposal from future iterations. Otherwise, the subproblem is checked for additional violated conflicts, which are added incrementally.

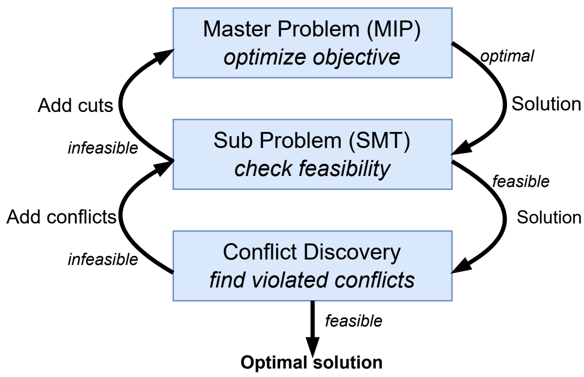


Figure 8: `CD-LBBD` workflow. The master MIP controls the coordination times for critical events and proposes candidate values. The subproblem checks microscopic feasibility (routing, ordering, conflicts) under these fixed times. If infeasible, a disjunctive feasibility cut is returned to exclude the master assignment; otherwise, conflict discovery checks for violated constraints and refines the subproblem until saturation. The process terminates with a provably optimal and feasible solution.

This decomposition structure is inspired by the feasibility-checking framework of Leutwiler and Corman (2022), but extends it in two important ways. First, we operate in a non-periodic, continuous-time setting, with routing choices and resource usage fully absorbed into the subproblem. Second, we apply lazy discovery of precedence disjunctions instead of enforcing all conflicts upfront. This limits the number of binary decisions and accelerates solving, especially on dense or large networks with many irrelevant conflicts.

7. Computational Study

This section covers a comprehensive computational study. Section 7.1 defines the case study based on the 2025 DISPLIB competition in train scheduling. Section 7.2 covers computational experiments and Section 7.3 analyzes the benefits and limitations of the proposed methods.

7.1. Case study: 2025 DISPLIB Competition

We present a case study based on the train dispatching benchmark library developed for the 2025 DISPLIB competition (Kloster et al., 2025). The library consists of 112 instances based on real-life networks. Table 2 shows a summary of the instance characteristics. For further details on the instances, the authors refer to Kloster et al. (2025).

Table 2: DISPLIB benchmark library.

Instance set	**# Instances**	**# Trains**		**# Operations**		**# Resources**	
		Min	Max	Min	Max	Min	Max
smi_close	9	5	13	113	1975	87	787
smi_headway	12	5	13	113	2245	87	790
nor1_full	10	4	16	148	796	82	95
nor1_critical	10	40	155	2194	8217	95	95
nor2	5	23	23	1448	8217	95	167
nor3	5	21	22	1237	1750	79	137
nor4_small	5	157	168	16 034	38 125	408	408
nor4_large	5	171	365	1237	38 125	79	408
nor5_small	5	261	286	25 594	50 934	512	512
nor5_large	5	346	505	16 034	50 934	408	512
swi	9	4	467	326	52 741	112	1301
wab_small	16	29	30	3234	3488	136	136
wab_large	16	44	48	5588	6054	136	136
TOTAL	112	4	505	113	52 741	79	1301

All methods have been implemented in Python using the *OpenBus* framework (Fuchs and Corman, 2019), with Gurobi 12.0.2 as the MIP solver (Gurobi Optimization, LLC, 2025). All experiments were conducted on a server with 8 CPU cores (Intel Xeon Gold 6248) and 32 GB of RAM. We run both methods in parallel, dedicating one thread for the LBBD and seven for the MIP approach. These resource limitations are derived from the DISPLIB 2025 competition (Kloster et al., 2025).

7.2. Main Results and Analysis

To measure the performance of our solution methods, we run them using two time limits: 10 minutes, as required in the DISPLIB 2025 competition, and 1 hour.

Tables 3 and 4 summarize the results across the instance sets with a time limit of 10 minutes and 1 hour, respectively. A detailed view of the results per individual instance is shown in Appendix B.

Table 3: Summary of computational results with a time limit of 10 minutes. "-" denotes that the method failed to build the model within the time limit. For a fair comparison, if `SC-MIP` fails to find an incumbent solution, we denote it with an optimality gap of 100 %.

	`CD-LBBD`						`SC-MIP`				**Combined**	
Instance Set	**# Opt. Sol.**	**Time (s)**	**# Cuts**	**# Conflicts**	**# Benders iterations**	**# Conflict disc. iterations**	**# Opt. Sol.**	**Opt. Gap (%)**	**Time (s)**	**# Conflicts**	**# Opt. Sol.**	**Opt. Gap (%)**
smi_close	9/9	127	16	26	160	8	7/9	6.7	147	531	9/9	0.0
smi_headway	7/12	280	34	52	203	10	8/12	5.6	234	2322	8/12	4.0
nor1_full	10/10	53	34	366	44	20	7/10	13.3	288	257 835	10/10	0.0
nor1_critical	10/10	5	18	72	26	14	10/10	0.0	7	1169	10/10	0.0
nor2	5/5	217	296	229	343	31	5/5	0.0	20	10 447	5/5	0.0
nor3	5/5	108	167	307	268	39	5/5	0.0	15	9636	5/5	0.0
nor4_small	5/5	184	69	595	67	31	0/5	67.3	600	225 562	5/5	0.0
nor4_large	4/5	395	136	669	104	30	0/5	87.0	600	385 063	4/5	16.8
nor5_small	0/5	600	352	1597	164	37	0/5	100.0	600	-	0/5	100.0
nor5_large	0/5	600	220	1376	63	21	0/5	100.0	600	-	0/5	100.0
swi	9/9	91	0	3412	0	29	9/9	0.0	130	86 359	9/9	0.0
wab_small	0/16	600	517	483	146	21	0/16	68.5	600	33 881	0/16	59.1
wab_large	0/16	600	422	600	143	20	0/16	68.8	600	95 525	0/16	68.1
Total	64/112						51/112	37.8			65/112	28.3

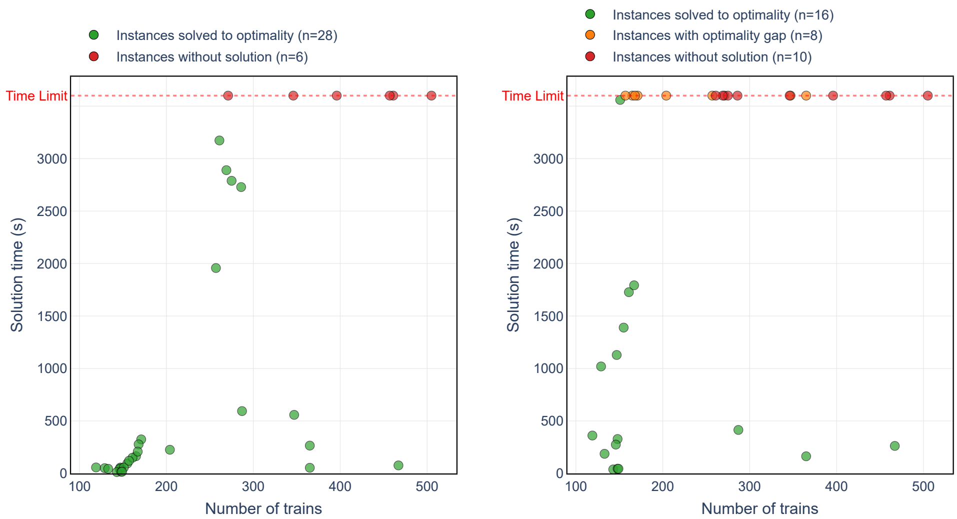
Table 4: Summary of computational results with a time limit of 1 hour. The reported results are averaged over the instances in an instance set. "-" denotes that the method failed to build the model within the time limit. For a fair comparison, if `SC-MIP` fails to find an incumbent solution, we denote it with an optimality gap of 100 %.

	`CD-LBBD`						`SC-MIP`				**Combined**	
Instance Set	**# Opt. Sol.**	**Time (s)**	**# Cuts**	**# Conflicts**	**# Benders iterations**	**# Conflict disc. iterations**	**# Opt. Sol.**	**Opt. Gap (%)**	**Time (s)**	**# Conflicts**	**# Opt. Sol.**	**Opt. Gap (%)**
smi_close	9/9	127	16	26	160	8	8/9	3.4	516	531	9/9	0.0
smi_headway	8/12	1,328	50	119	403	15	8/12	4.8	1223	2338	9/12	2.5
nor1_full	10/10	53	34	366	44	20	10/10	0.0	844	257 835	10/10	0.0
nor1_critical	10/10	5	18	72	26	14	10/10	0.0	7	1169	10/10	0.0
nor2	5/5	217	296	229	343	31	5/5	0.0	19	10 447	5/5	0.0
nor3	5/5	108	167	307	268	39	5/5	0.0	20	9636	5/5	0.0
nor4_small	5/5	184	69	595	67	31	2/5	2.0	2856	225 859	5/5	0.0
nor4_large	5/5	666	240	713	220	36	0/5	49.5	3587	384 988	5/5	0.0
nor5_small	4/5	3036	568	2321	388	77	0/5	100.0	3600	-	4/5	20.0
nor5_large	0/5	3600	706	2620	256	69	0/5	100.0	3600	-	0/5	100.0
swi	9/9	91	0	3412	0	29	9/9	0.0	130	86 359	9/9	0.0
wab_small	0/16	3600	913	733	214	51	0/16	56.5	3600	33 881	0/16	45.6
wab_large	0/16	3600	585	711	132	30	0/16	63.9	3600	95 525	0/16	63.1
Total	70/112						57/112	29.3			71/112	24.7

The results show the excellent scalability of both methods. The `CD-LBBD` can solve 64 instances to optimality within 10 minutes and up to 70 within 1 hour. The `SC-MIP` method solves 51 instances within 10 minutes and 57 instances within 1 hour, while significantly reducing the optimality gap in the hardest instances. Interestingly, none of the methods dominates the other one, as they achieve optimal solutions in different instances. Together, they find 65 optimal solutions within 10 minutes, and 70 within 1 hour, highlighting the value of considering both methods together. More interestingly, the tighter lower bound of `CD-LBBD`, combined with the best incumbent solutions from `SC-MIP`, helps further reduce the optimality gap in instances that failed to converge, thereby consolidating the quality of the solution. Both methods fail to find solutions to the largest instances (`nor5_large`) within the time limit. It should be noted that these instances are considerably large and cover most of the Norwegian railroad network (i.e., Oslo region) for a full day of operations (Kloster et al., 2025). The extension of the time limit from 10 minutes to 1 hour especially improves the solutions to the `nor4_large` and `nor5_small` instances. It however does not provide much benefit for achieving or proving optimality in the `wab` instances which are characterized by high routing flexibility and large delays.

Both methods also leverage the network topologies differently. `CD-LBBD` only needs to consider a fraction of potential conflicts in a problem and aggregates the information into a small number of cuts for the master, helped by dominance filtering as explained in Section 6.2. The `SC-MIP` approach provides feasible solutions for all instances except `nor5` by iteratively expanding the solution search space around unresolved conflicts. The complementary nature of the two methods becomes evident when examining their performance characteristics across different instance types. Figure 9 compares the performance of both methods on instances with more than 100 trains. We observe that `CD-LBBD` scales well with the number of trains, solving all instances with up to 200 trains to optimality within one hour, whereas `SC-MIP` requires more time for such instances and fails to reach optimality for several of them. Conversely, Figure 10 indicates that `SC-MIP` performs better on instances with high routing flexibility, measured here by the median number of alternative paths from source to sink across all trains in an instance. On instances with a large number of routing alternatives, `SC-MIP` consistently obtains feasible solutions. In contrast, `CD-LBBD` fails to find solutions for most instances but does provide tighter lower bounds. This observed complementarity, where `CD-LBBD` solves larger instances with limited routing flexibility to optimality, and `SC-MIP` provides feasible solutions for instances with extensive routing alternatives, motivates the hybrid framework and demonstrates the value of combining both methods to achieve broader coverage across the benchmark.

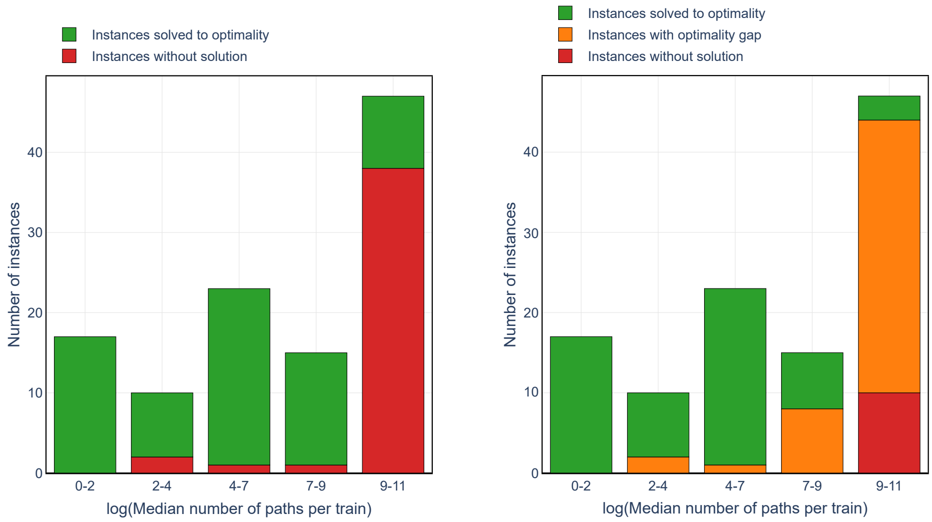
These results position our hybrid method competitively within the DISPLIB 2025 benchmark[2]. The approach submitted to the competition (*openbus*) identified 80 best-known solutions out of 112 instances. Subsequent refinements, including minor algorithmic adjustments and bug fixes, improved this to 85 best-known solutions within the competition time limit and to 91 in total (81 % of the benchmark), contributing to the state-of-the-art in microscopic train timetabling.



(a) Performance of `CD-LBBD`. (b) Performance of `SC-MIP`.

Figure 9: Comparison of solution times for `CD-LBBD` and `SC-MIP` versus number of trains for instances with more than 100 trains (1 hour time limit).

[2] `https://displib.github.io/`



(a) `CD-LBBD`: solved/unsolved instances

(b) `SC-MIP`: optimal/suboptimal solutions

Figure 10: Solution outcomes versus routing flexibility, measured by the median number of paths per train in an instance on a log-scale (1 hour time limit).

To better understand the contribution of each algorithmic component, we measure the value of conflict discovery (`CD`) in the LBBD method by comparing our proposed `CD-LBBD` with a standard LBBD approach without `CD`, denoted as `AC-LBBD` (All-Conflicts-LBBD). Similarly, we compare the `SC-MIP` approach with an equivalent one that handles hard conflicts (`HC-MIP`). For simplicity, the remaining experiments in this section (i.e., Section 7) are conducted with the competition time limit of 10 minutes.

Table 5: Comparison of performance of LBBD with (`CD-LBBD`) and without (`AC-LBBD`) conflict discovery, and the MIP-based approach with soft (`SC-MIP`) and hard (`HC-MIP`) conflicts with a 10 minute time limit. For a fair comparison, if `HC-MIP` or `SC-MIP` fail to find an incumbent solution, we denote it with an optimality gap of 100 %.

	`AC-LBBD`		`CD-LBBD`		`HC-MIP`			`SC-MIP`		
Instance set	**# Opt. Sol.**	**Time (s)**	**# Opt. Sol.**	**Time (s)**	**# Opt. Sol.**	**Opt. Gap (%)**	**Time (s)**	**# Opt. Sol.**	**Opt. Gap (%)**	**Time (s)**
smi_close	8/9	171	9/9	127	6/9	7.1	249	7/9	6.7	147
smi_headway	8/12	227	7/12	280	7/12	6.9	259	8/12	5.6	234
nor1_full	2/10	514	10/10	53	4/10	60.0	461	7/10	13.3	288
nor1_critical	10/10	5	10/10	5	10/10	0.0	5	10/10	0.0	7
nor2	5/5	130	5/5	217	5/5	0.0	41	5/5	0.0	20
nor3	5/5	71	5/5	109	5/5	0.0	33	5/5	0.0	15
nor4_small	0/5	600	5/5	184	0/5	100.0	600	0/5	67.3	600
nor4_large	0/5	600	4/5	395	0/5	100.0	600	0/5	87.0	600
nor5_small	0/5	600	0/5	600	0/5	100.0	600	0/5	100.0	600
nor5_large	0/5	600	0/5	600	0/5	100.0	600	0/5	100.0	600
swi	9/9	64	9/9	91	9/9	0.0	104	9/9	0.0	130
wab_small	0/16	600	0/16	600	0/16	100.0	600	0/16	68.5	600
wab_large	0/16	600	0/16	600	0/16	100.0	600	0/16	68.8	600
Total	47/112		64/112		46/112	53.1		51/112	37.8	

The results in Table 5 show that both conflict discovery and soft-conflict handling provide an overall advantage to the methods, solving more instances to optimality and achieving more incumbent solutions and tighter optimality gaps in the case of `SC-MIP`. Interestingly, we observe that `AC-LBBD` finds one optimal solutions which `CD-LBBD` does not (`smi_headway_1`) and is faster on some instances, however, it performs much worse in others (e.g., it fails to solve most of the `nor1_full` and `nor4` instances), which leads to an overall poorer performance. In general, one can see that the average solving time for the `smi_headway` and `swi` instances is lower for the `AC-LBBD` than the `CD-LBBD`. For the `smi` instances, this can be attributed to the smaller number of potential conflicts (on the order of 10^3) in these instances compared to the `nor1_full` instances (on the order of 10^5). Whereas for the `swi` instances, as the objective value is 0, only a single sub-problem needs to be solved in the `AC-LBBD` implementation, compared to a sub-problem for every iteration of the conflict discovery in `CD-LBBD`.

In the case of `SC-MIP`, it performs similarly to `HC-MIP` in the fastest instances. Still, it clearly dominates the rest, indicating that soft conflicts are especially important for finding feasible solutions as a starting point for optimizing more challenging instances.

7.3. Scalability by Strengthening

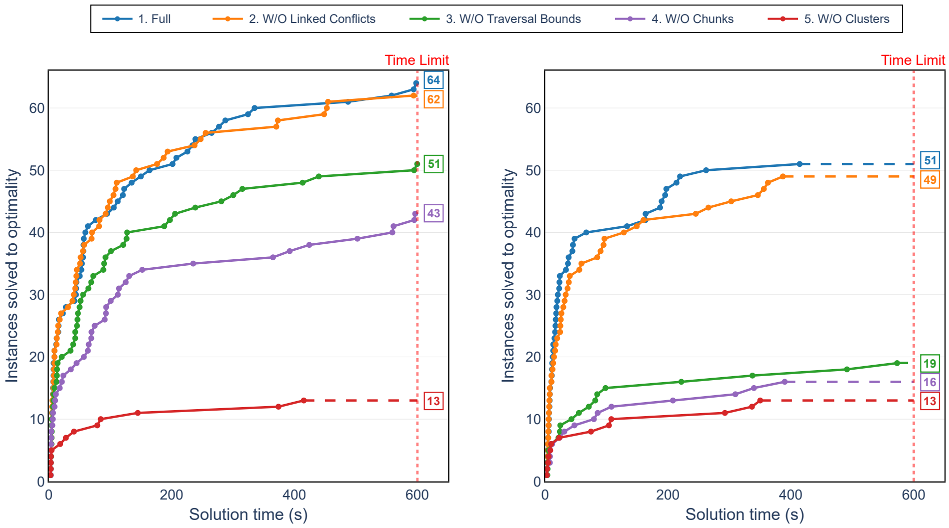
In this section we study the value of the modeling enhancements described in Section 4. To measure the value of each strengthening technique, we compare variants of the proposed method, adding one technique at a time, as the natural implementation of the techniques is sequential by nature. Starting from the presented model with all strengthening techniques, we remove and compare one method in the following order: (i) linked conflicts, (ii) traversal bounds, (iii) chunks, and (iv) clusters. Time window propagation is applied to all variants as a standard and computationally inexpensive preprocessing step.

Table 6 summarizes the different aspects used to strengthen the model.

Table 6: Average values per instance set of modeling characteristics used in strengthening techniques.

				Conflict chunking			**Variable clustering**		
Instance set	**# linked conf.**	**# cluster tr.b.**	**# route tr.b.**	**# before**	**# after**	**% reduct.**	**# before**	**# after**	**% reduct.**
smi_close	42	341	13	39 545	531	−91.2	1325	488	−60.2
smi_headway	88	394	15	37 156	2071	−85.5	1421	542	−59.8
nor1_full	12 528	3305	1526	485 241	257 835	−47.2	6422	4036	−37.1
nor1_critical	74	242	109	2439	1169	−51.5	474	305	−35.5
nor2	676	707	305	30 244	10 447	−65.4	1609	922	−42.7
nor3	910	639	267	20 415	9636	−52.8	1311	855	−34.8
nor4_small	6231	8142	3810	2 393 085	765 103	−68.0	17 402	9022	−48.2
nor4_large	17 959	13 031	6139	7 238 377	2 336 507	−67.8	28 094	14 364	−48.9
nor5_small	3860	12 646	6061	4 850 751	1 500 954	−69.0	27 217	13 856	−49.1
nor5_large	10 852	20 322	9738	12 321 771	3 949 489	−68.0	43 849	22 291	−49.2
swi	863 939	1190	124	14 891 565	496 421	−93.0	17 049	14 658	−9.1
wab_small	249	1656	730	76 653	33 881	−55.8	3452	2333	−32.4
wab_large	703	2755	1212	229 343	95 530	−58.4	5848	3876	−33.7
Average	72 505	3589	1611	464 453	2 489 927	−66.6	5287	8907	−40.1

We observe that the number of (potential) conflicts and variables is correlated with the instance size (i.e., number of trains and operations), ranging from a few thousand to more than 14 million. In terms of variables, the range is smaller, with instances including a few hundred to more than 43 000. These two modeling aspects are the main ones tackled by the strengthening techniques. In fact, grouping conflicts into chunks allows for reducing the number of conflicts by an average of 66 %, with this reduction reaching 93 % in some cases. Similarly, variable clustering can significantly reduce the number of problem variables, by an average of 40 %, and more than 60 % in some cases. The number of traversal bounds varies by instance; however, as one would expect, there are usually more clusters than route traversal bounds. Finally, linking conflicts enables the removal of binary decision variables due to the combined consideration of precedences. The potential to link conflicts is strongly dependent on the instance set, with the most drastic effects in the `swi` instances, where the average of linked conflicting chunks exceeds the number of potential conflicts. While it may be intuitively assumed that these model-size reductions are valuable for the proposed methods, as they should enhance their solving capabilities, we proceed to assess their effect by sequentially removing them.



(a) Performance of `CD-LBBD`.

(b) Performance of `SC-MIP`.

Figure 11: Performance of approaches on instances solved to optimality when successively removing reductions and strengthenings. Every point is an instance solved to optimality in that solution time.

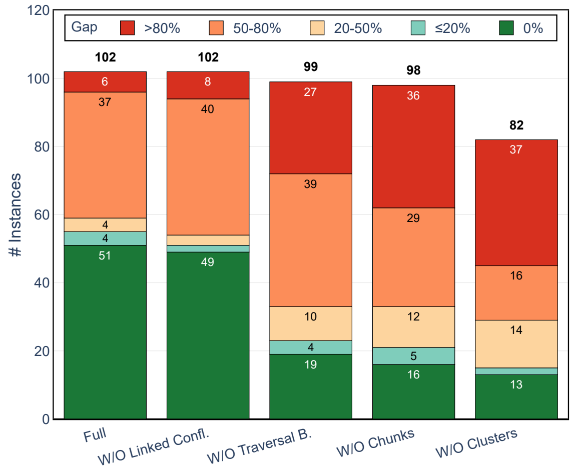


Figure 12: Gap of `SC-MIP` over the instances after solving for 600 s when successively removing reductions and strengthenings. The total height of the bar is the number of instances for which a solution was found.

The results show that all presented techniques make a significant contribution to scaling the methodology. Figures 11a and 11b show the number of instances solved to optimality over time when removing one strengthening technique after the other for the `SC-MIP` and `CD-LBBD`, respectively. Since these preprocessing steps exhibit dependencies as described in Section 4, we evaluate them by removing one technique at a time in a feasible order. The fully strengthened model not only solves more instances to proven optimality but also does it in considerably shorter time. We observe that all techniques are valuable, but particularly *traversal bounds* for `SC-MIP` and *clustering* for `CD-LBBD`. Furthermore, the techniques prove to be generalizable as they improve the performance of the two presented methods, which are notably different in nature. Figure 12 shows a more detailed view of the solution improvement when incorporating strengthening techniques in `SC-MIP`. Not only are more optimal solutions found, but the optimality gap of the remaining ones also improves significantly.

8. Conclusion and Outlook

This study presents a hybrid framework for microscopic railway timetabling. The method supports tactical and operational timetabling as introduced in a practice-oriented process model for timetabling. The framework combines two complementary exact optimization methods: a Soft-Conflict Mixed-Integer Programming (`SC-MIP`) approach that guides the search toward feasibility through iterative relaxation, and a Logic-Based Benders Decomposition with Conflict Discovery (`CD-LBBD`) framework that separates coordination decisions from microscopic feasibility checking. Both approaches operate on a strengthened continuous-time event-activity formulation, for which we introduce a comprehensive set of preprocessing and reduction techniques, including bound propagation, topological clustering, traversal bounds, resource aggregation into chunks, and conflict linking. The proposed design leverages decomposition and relaxation to strike a balance between modeling fidelity and computational tractability.

Computational experiments on the 112 instances of the DISPLIB 2025 benchmark demonstrate the effectiveness and complementarity of the two methods. The hybrid method solves 65 instances to proven optimality within 10 minutes and up to 71 within an hour, contributing 91 best-known solutions to the benchmark. The ablation study confirms that each strengthening technique provides measurable gains, with variable clustering and traversal-bounds proving particularly impactful. Together, the strengthening techniques enable solving nearly three times as many instances to proven optimality compared to the baseline formulation.

Furthermore, the two methods exhibit complementary strengths across instances characteristics. `CD-LBBD` scales effectively with the number of trains, solving instances with up to 350 trains to optimality by lazily discovering conflicts and compactly encoding the structural information to guide the optimization. `SC-MIP` excels on instances with high routing flexibility, consistently finding feasible solutions by iteratively expanding the search space around unresolved conflicts.

The findings indicate that combining decomposition-based and relaxation-based exact methods can substantially improve both solution quality and robustness for large-scale microscopic timetabling problems. Remaining challenges concern increasing scalability to tackle the largest instances which suggest further research into spatial or temporal decomposition strategies. Furthermore, future research directions may extend the hybridization between the two methods, sharing information, such as feasible solutions or Benders cuts. Learning mechanisms that anticipate conflicts or structural bottlenecks ahead of time may enhance the methods as well, for example identifying areas of infeasibility in constrained time windows. During solving itself, managing the number of Benders cuts and discovered conflicts may improve the master- and subproblem solution time. These directions open promising opportunities for advancing real-time, exact optimization methods in tactical and operational railway timetabling.

Acknowledgements

This research was partly supported by the Swiss National Science Foundation (SNSF) through the National Centre of Competence in Research (NCCR) and by the Center for Sustainable Future Mobility (CSFM) through the "Learning railways for better schedules (LeRaBe)" project. The authors thank our long-term collaborators at the Swiss Federal Railways (SBB), and particularly Manuel Kohler and Jürg Krähenbühl for insightful discussions. Finally, we thank the DISPLIB organizers for arranging the competition.

References

Armando, A., Castellini, C., Giunchiglia, E., and Maratea, M. (2004). A sat-based decision procedure for the boolean combination of difference constraints. In *International Conference on Theory and Applications of Satisfiability Testing*, pages 16–29. Springer.

Borndörfer, R., Lindner, N., and Roth, S. (2020). A concurrent approach to the periodic event scheduling problem. *Journal of Rail Transport Planning and Management*, 15:140–159.

Bussieck, M. R., Winter, T., and Zimmermann, U. T. (1997). Discrete optimization in public rail transport. *Mathematical Programming*, 79(1-3):415–444.

Cacchiani, V., Huisman, D., Kidd, M., Kroon, L., Toth, P., Veelenturf, L., and Wagenaar, J. (2014). An overview of recovery models and algorithms for real-time railway rescheduling. *Transportation Research Part B: Methodological*, 63:15–37.

Caimi, G., Chudak, F., Fuchsberger, M., Laumanns, M., and Zenklusen, R. (2011). A new resource-constrained multicommodity flow model for conflict-free train routing and scheduling. *Transportation science*, 45(2):212–227.

Caimi, G., Kroon, L., and Liebchen, C. (2017). Models for railway timetable optimization: Applicability and applications in practice. *Journal of Rail Transport Planning & Management*, 6(4):285–312.

Corman, F., D'Ariano, A., Pacciarelli, D., and Pranzo, M. (2014). Dispatching and coordination in multi-area railway traffic management. *Computers & Operations Research*, 44:146–160.

D'Ariano, A., Pacciarelli, D., and Pranzo, M. (2007). A branch and bound algorithm for scheduling trains in a railway network. *European Journal of Operational Research*, 183(2):643–657.

Fuchs, F. and Corman, F. (2019). An open toolbox for integrated optimization of public transport. In *2019 6th International Conference on Models and Technologies for Intelligent Transportation Systems (MT-ITS)*, pages 1–7.

Fuchs, F., Martin-Iradi, B., and Corman, F. (2025a). Optimizing periodic stability in railway timetables: A microscopic model for networks with a macroscopic comparison. *Journal of Rail Transport Planning & Management*, 36:100555.

Fuchs, F., Martin-Iradi, B., and Corman, F. (2025b). Solving Integrated Periodic Railway Timetabling with Satisfiability Modulo Theories: A Scalable Approach to Routing and Vehicle Circulation. arXiv:2507.11489 [math].

Fuchs, F., Trivella, A., and Corman, F. (2022). Enhancing the interaction of railway timetabling and line planning with infrastructure awareness. *Transportation Research Part C: Emerging Technologies*, 142:103805.

Gurobi Optimization, LLC (2025). Gurobi Optimizer Reference Manual.

Hooker, J. N. and Ottosson, G. (2003). Logic-based benders decomposition. *Mathematical Programming*, 96(1):33–60.

Kloster, O., Luteberget, B., Mannino, C., and Sartor, G. (2025). Displib: a library of train dispatching problems.

Kroon, L. G., Zuidwijk, R. A., and van Hoesel, S. J. C. (2009). The new dutch timetable: The OR revolution. *Interfaces*, 39(4):6–17.

Lamorgese, L. and Mannino, C. (2015). An exact decomposition approach for the real-time train dispatching problem. *Operations Research*, 63(1):48–64.

Lamorgese, L., Mannino, C., and Piacentini, M. (2016). Optimal train dispatching by benders'-like reformulation. *Transportation Science*, 50(3):910–925.

Leutwiler, F., Bonet Filella, G., and Corman, F. (2023). Accelerating logic-based Benders decomposition for railway rescheduling by exploiting similarities in delays. *Computers & Operations Research*, 150:106075.

Leutwiler, F. and Corman, F. (2022). A logic-based benders decomposition for microscopic railway timetable planning. *European Journal of Operational Research*, 303(2):525–540.

Leutwiler, F. and Corman, F. (2023). A review of principles and methods to decompose large-scale railway scheduling problems. *EURO Journal on Transportation and Logistics*, page 100107.

Liebchen, C. and Möhring, R. H. (2007). The modeling power of the periodic event scheduling problem: Railway timetables — and beyond. In Geraets, F., Kroon, L., Schoebel, A., Wagner, D., and Zaroliagis, C. D., editors, *Algorithmic Methods for Railway Optimization*, volume 4359 of *Lecture Notes in Computer Science*, pages 3–40. Springer Berlin Heidelberg.

Mannino, C. and Nakkerud, A. (2023). Optimal train rescheduling in oslo central station. *Omega*, 116:102796.

Pascariu, B., Samà, M., Pellegrini, P., D'Ariano, A., Rodriguez, J., and Pacciarelli, D. (2022). Effective train routing selection for real-time traffic management: Improved model and aco parallel computing. *Computers & Operations Research*, 145:105859.

Peeters, L. W. P. (2003). *Cyclic railway timetable optimization*, volume 22. Erasmus University Rotterdam, Roterdam.

Pellegrini, P., Marlière, G., Pesenti, R., and Rodriguez, J. (2015). Recife-milp: An effective milp-based heuristic for the real-time railway traffic management problem. *IEEE Transactions on Intelligent Transportation Systems*, 16(5):2609–2619.

Pellegrini, P., Pesenti, R., and Rodriguez, J. (2019). Efficient train re-routing and rescheduling: Valid inequalities and reformulation of RECIFE-MILP. *Transportation Research Part B: Methodological*, 120:33–48.

Samà, M., D'Ariano, A., Pacciarelli, D., and Corman, F. (2016). Lower and upper bound algorithms for the real-time train scheduling and routing problem in a railway network. *IFAC-PapersOnLine*, 49(3):215–220. 14th IFAC Symposium on Control in Transportation SystemsCTS 2016.

SBB Infrastruktur / SBB Cargo (2014). So entsteht der fahrplan: Fahrplaninfrastruktur. `https://blog.sbbcargo.com/wp-content/uploads/2014/11/A2_Plakat_Fahrplan_Infrastruktur_dt_Web.pdf`. April 2014, Faltprospekt.

Schlechte, T., Borndörfer, R., Erol, B., Graffagnino, T., and Swarat, E. (2011). Micro–macro transformation of railway networks. *Journal of Rail Transport Planning & Management*, 1(1):38–48.

Varga, J., Karlsson, E., Raidl, G. R., Rönnberg, E., Lindsten, F., and Rodemann, T. (2023). Speeding up logic-based benders decomposition by strengthening cuts with graph neural networks. In *International Conference on Machine Learning, Optimization, and Data Science*, pages 24–38. Springer.

Appendix A. Linearized model

This appendix presents a linearized MIP equivalent of the compact disjunctive model in Section 5. We keep the same sets, variables, and parameters. Bounds used below are those obtained after bound propagation and clustering (Section 4). For any event $e \in \mathcal{E}$, let $[lb_e, ub_e]$ be its propagated time window; for any chunk $q \in Q$, let $[\underline{\tau}_q^{\text{acq}}, \overline{\tau}_q^{\text{acq}}]$ and $[\underline{\tau}_q^{\text{rel}}, \overline{\tau}_q^{\text{rel}}]$ be valid bounds on acquisition and release times derived in Section 4.4; and for any activity $a = (e, e')$ let $[lb_a, ub_a]$ denote its duration window. Traversal bounds $d_{ee'}^{\min}$ are as in Equation (M3).

Objective Function: unchanged from Equation (M0)

$$\min \sum_{e \in O} \Big(c_e \cdot \max\{0, t_e - \bar{t}_e\} + d_e \cdot H(t_e - \bar{t}_e) \Big) \tag{LM0}$$

if a fully linear objective is required, $\max\{\cdot\}$ and $H(\cdot)$ can be replaced by a linear epigraph and binary threshold variables.

Route selection constraints: linearization of Equation (M1)

$$\sum_{w \in \rho^+(e)} x_w - \sum_{w \in \rho^-(e)} x_w = \begin{cases} 1 & e \in \mathcal{E}_z^{\text{source}} \\ -1 & e \in \mathcal{E}_z^{\text{sink}} \\ 0 & \text{otherwise} \end{cases} \qquad \forall e \in \mathcal{E}^{\text{TFN}} \tag{LM1}$$

Conditional timing constraints: linearization of Equation (M2)

For each $a = (e, e') \in \mathcal{A}$ with $w = \alpha(a)$ and binary x_w:

$$t_{e'} - t_e \geq lb_a - M_a^{\downarrow}(1 - x_w) \tag{LM2a}$$

$$t_{e'} - t_e \leq ub_a + M_a^{\uparrow}(1 - x_w) \tag{LM2b}$$

with tight coefficients:

$$M_a^{\downarrow} = lb_a - (lb_{e'} - ub_e) \qquad M_a^{\uparrow} = (ub_{e'} - lb_e) - ub_a$$

Always-active traversal bounds: unchanged from Equation (M3)

$$t_{e'} - t_e \geq d_{ee'}^{\min} \qquad \forall (e, e') \in \mathcal{B}. \tag{LM3}$$

Chunk activation and timing: linearization of Equations (M4a) and (M4b)

One-sided timing enclosures:

$$\tau_q^{\text{acq}} - t_{e^{\text{in}}(w)} \leq M_{q,w}^{\text{acq}} (1 - x_w) \qquad \forall q \in Q,\ w \in \mathcal{W}_q^{\text{in}} \tag{LM4a}$$

$$t_{e^{\text{out}}(w)} - \tau_q^{\text{rel}} \leq M_{q,w}^{\text{rel}} (1 - x_w) \qquad \forall q \in Q,\ w \in \mathcal{W}_q^{\text{out}} \tag{LM4b}$$

with tight coefficients:

$$M_{q,w}^{\text{acq}} = \overline{\tau}_q^{\text{acq}} - lb_{e^{\text{in}}(w)} \qquad M_{q,w}^{\text{rel}} = ub_{e^{\text{out}}(w)} - \underline{\tau}_q^{\text{rel}}$$

Conflict resolution over resource chunks linearization of Equations (M5a) and (M5b)

Activation coupling:

$$x_q \geq x_w \qquad \forall q \in \mathcal{Q},\ w \in \mathcal{W}_q \tag{LM5a}$$

Introduce precedence binaries $y_{qq'}, y_{q'q} \in \{0, 1\}$ for each conflicting pair (q, q') indicating the precedence choice $q \prec q'$ and $q' \prec q$ respectively, with activation guards:

$$y_{qq'} + y_{q'q} \leq x_q \qquad y_{qq'} + y_{q'q} \leq x_{q'} \tag{LM5-aux1}$$

$$y_{qq'} + y_{q'q} \geq x_q + x_{q'} - 1 \tag{LM5-aux2}$$

Big-M separation inequalities:

$$\tau_{q'}^{\text{acq}} - \tau_q^{\text{rel}} \geq \delta(r(q), q) - M_{qq'}^{\text{conf}}(1 - y_{qq'}) \tag{LM5b-qq'}$$

$$\tau_q^{\text{acq}} - \tau_{q'}^{\text{rel}} \geq \delta(r(q'), q') - M_{q'q}^{\text{conf}}(1 - y_{q'q}) \tag{LM5b-$q'q$}$$

with tight coefficients:

$$M_{qq'}^{\text{conf}} = (\overline{\tau}_q^{\text{rel}} - \underline{\tau}_{q'}^{\text{acq}}) + \delta(r(q), q) \qquad M_{q'q}^{\text{conf}} = (\overline{\tau}_{q'}^{\text{rel}} - \underline{\tau}_q^{\text{acq}}) + \delta(r(q'), q')$$

Note: If a single selector $y_{qq'} \in \{0, 1\}$ with $y_{q'q} = 1 - y_{qq'}$ is preferred, replace the guards above by $y_{qq'} + y_{q'q} = x_q + x_{q'} - 1$. However, this is only valid if both resource chunks cannot be bypassed; otherwise, one might over-restrict the problem.

Cluster substitution constraints: linearization of Equations (M6a) and (M6b)

Introduce event-activation binaries $z_{C,e} \in \{0, 1\}$ for $e \in C$, representing the realized event of cluster C:

$$z_{C,e} \geq x_w \qquad \forall C \in \mathcal{C},\ e \in C,\ w \in \rho^-(e) \tag{LM6-aux}$$

$$t_e - t_C \leq M_{C,e}^{\text{cl}}(1 - z_{C,e}) \qquad t_C - t_e \leq M_{C,e}^{\text{cl}}(1 - z_{C,e}) \tag{LM6a}$$

$$lb_e\, z_{C,e} \leq t_C \leq ub_e\, z_{C,e} + M_{C,e}^{\text{cb}}(1 - z_{C,e}) \tag{LM6b}$$

with tight coefficients:

$$M_{C,e}^{\text{cl}} = \max\{ub_e - lb_C,\ ub_C - lb_e\} \qquad M_{C,e}^{\text{cb}} = ub_C$$

where $[lb_C, ub_C]$ are the cluster bounds defined in Section 4.2.

Variable domains: unchanged from Equations (M7a) to (M7e)

No new continuous variables are introduced, and all auxiliary activation and precedence indicators $(y_{qq'}, z_{C,e})$ are binary.

Appendix B. Detailed results

Table B.7: Results on `smi` instances with a time limit of 10 minutes. Bold values indicate that `CD-LBBD` found the better lower bound.

	CD-LBBD						SC-MIP			
Instance	**LB**	**Time (s)**	**# Cuts**	**# Conflicts**	**# Benders iterations**	**# Conflict disc. iter.**	**UB**	**LB**	**Time (s)**	**# Conflicts**
smi_close_0	679	4	5	10	7	4	681	681	5	122
smi_close_1	**4,316**	487	40	64	617	14	4,321	2,499	600	752
smi_close_2	1,331	106	29	64	151	9	1,338	1,338	37	659
smi_close_3	1,860	5	6	6	2	2	1,866	1,866	11	554
smi_close_4	24,225	3	3	5	3	3	24,228	24,228	2	10
smi_close_5	314	287	11	7	107	9	315	315	48	1,047
smi_close_6	21,034	7	9	27	48	13	21,040	21,040	6	508
smi_close_7	**582**	239	29	20	483	11	586	479	600	359
smi_close_8	3,413	8	11	34	18	6	3,419	3,419	11	772
smi_headway_0	1,483	9	7	13	80	4	1,483	1,483	5	154
smi_headway_1	**4,543**	600	39	73	591	11	5,040	3,655	600	5,488
smi_headway_2	2,302	600	27	64	272	7	2,897	2,629	600	1,957
smi_headway_3	3,552	9	10	12	23	5	3,552	3,552	20	1,312
smi_headway_4	24,797	3	3	5	3	3	24,797	24,797	3	10
smi_headway_5	**667**	600	8	14	20	8	868	659	600	3,932
smi_headway_6	22,236	7	10	29	53	16	22,236	22,236	10	1,030
smi_headway_7	1,150	201	31	22	322	10	1,150	1,150	133	644
smi_headway_8	5,159	23	18	70	80	10	5,159	5,159	17	1,266
smi_headway_9	16,006	112	53	109	397	17	16,006	16,006	34	4,002
smi_headway_10	8,313	600	183	86	557	26	9,007	8,484	600	4,358
smi_headway_11	2,222	600	18	121	32	7	5,579	5,579	190	3,708

Table B.8: Results on `nor` instances with a time limit of 10 minutes. Bold values indicate that `CD-LBBD` found the better lower bound. Empty values mean that the method failed to build the model within the time limit.

Instance	CD-LBBD						SC-MIP			
	LB	**Time (s)**	**# Cuts**	**# Conflicts**	**# Benders iterations**	**# Conflict disc. iter.**	**UB**	**LB**	**Time (s)**	**# Conflicts**
nor1_critical_0	4,133	4	12	65	16	14	4,140	4,140	7	1,612
nor1_critical_1	2,416	4	6	66	14	12	2,424	2,424	6	870
nor1_critical_2	3,775	4	11	62	10	12	3,779	3,779	6	970
nor1_critical_3	8,016	8	43	168	46	24	8,024	8,024	13	3,055
nor1_critical_4	1,506	3	2	9	3	5	1,508	1,508	4	75
nor1_critical_5	2,677	3	3	18	7	8	2,679	2,679	4	292
nor1_critical_6	4,491	6	38	78	50	15	4,497	4,497	8	1,514
nor1_critical_7	4,137	4	6	36	11	12	4,145	4,145	6	938
nor1_critical_8	3,836	8	41	84	68	17	3,851	3,851	7	1,043
nor1_critical_9	5,488	5	21	131	33	24	5,495	5,495	8	1,316
nor1_full_0	3,299	57	34	316	50	16	3,310	3,310	220	238,183
nor1_full_1	2,486	53	5	442	12	16	2,489	2,489	195	367,212
nor1_full_2	6,046	15	34	171	40	17	6,053	6,053	23	25,714
nor1_full_3	2,658	59	105	386	147	44	2,664	2,664	67	54,313
nor1_full_4	5,358	45	48	369	58	25	5,377	5,377	214	136,633
nor1_full_5	**3,732**	50	19	324	25	18	4,211	3,511	600	274,305
nor1_full_6	**5,262**	56	14	469	15	16	8,655	4,432	600	354,233
nor1_full_7	1,012	42	7	227	5	6	1,017	1,017	163	350,745
nor1_full_8	**5,370**	95	61	570	67	29	9,250	3,004	600	400,516
nor1_full_9	6,076	56	11	382	17	16	6,085	6,085	197	376,500
nor2_1	4,937	234	330	234	372	27	4,965	4,965	21	12,854
nor2_2	4,619	135	211	240	334	23	4,648	4,648	24	12,592
nor2_3	5,500	64	187	250	208	26	5,522	5,522	18	9,745
nor2_4	6,186	598	620	171	613	53	6,211	6,211	23	8,523
nor2_5	5,416	56	134	249	189	27	5,430	5,430	16	8,523
nor3_1	3,667	335	387	385	495	53	3,701	3,701	17	10,009
nor3_2	5,740	123	180	352	426	48	5,780	5,780	18	10,202
nor3_3	5,562	12	43	225	53	25	5,581	5,581	11	8,792
nor3_4	4,605	28	114	237	120	26	4,617	4,617	13	9,290
nor3_5	2,923	45	110	334	246	45	2,951	2,951	15	9,888
nor4_small_1	**8,972**	164	56	688	35	34	22,358	7,030	600	228,651
nor4_small_2	**11,217**	277	85	731	137	40	21,520	9,028	600	232,573
nor4_small_3	**7,924**	150	77	484	46	29	17,283	5,992	600	221,013
nor4_small_4	**8,189**	121	45	432	29	20	23,197	6,031	600	213,557
nor4_small_5	**12,144**	208	81	642	86	34	33,401	9,855	600	232,018
nor4_large_1	**9,551**	265	86	518	72	27	75,965	7,698	600	537,922
nor4_large_2	**8,439**	558	72	1,003	47	38	85,622	6,529	600	510,923
nor4_large_3	**11,985**	324	252	600	131	35	40,567	8,086	600	225,258
nor4_large_4	**12,836**	600	165	685	211	23	81,057	9,788	600	368,521
nor4_large_5	**8,043**	226	106	539	58	27	39,437	5,992	600	282,689
nor5_small_1	**57,035**	600	343	1,472	143	35				
nor5_small_2	**65,762**	600	407	1,333	176	43				
nor5_small_3	**57,072**	600	358	1,961	167	35				
nor5_small_4	**56,979**	600	334	1,852	162	36				
nor5_small_5	**62,325**	600	316	1,369	170	37				
nor5_large_1	**71,414**	600	370	1,310	128	34				
nor5_large_2	**58,910**	600	194	1,403	42	14				
nor5_large_3	**52,821**	600	135	1,089	32	16				
nor5_large_4	**36,829**	600	107	1,120	18	11				
nor5_large_5	**65,684**	600	295	1,960	95	28				

Table B.9: Results on `swi` instances with a time limit of 10 minutes. Bold values indicate that `CD-LBBD` found the better lower bound.

	CD-LBBD						SC-MIP			
Instance	**LB**	**Time (s)**	**# Cuts**	**# Conflicts**	**# Benders iterations**	**# Conflict disc. iter.**	**UB**	**LB**	**Time (s)**	**# Conflicts**
swi_1	0	3	0	-	0	1	0	0	3	266
swi_2	0	7	0	505	0	4	0	0	12	15,269
swi_3	0	13	0	1,226	0	7	0	0	38	48,965
swi_4	0	16	0	1,474	0	8	0	0	45	56,030
swi_5	0	16	0	1,462	0	8	0	0	44	56,031
swi_6	0	54	0	3,739	0	10	0	0	163	148,452
swi_7	0	76	0	6,172	0	10	0	0	262	260,791
swi_8	0	44	0	2,031	0	24	0	0	187	115,831
swi_9	0	594	0	14,097	0	192	0	0	414	75,594

Table B.10: Results on `wab` instances with a time limit of 10 minutes. Bold values indicate that `CD-LBBD` found the better lower bound.

	CD-LBBD						SC-MIP			
Instance	**LB**	**Time (s)**	**# Cuts**	**# Conflicts**	**# Benders iterations**	**# Conflict disc. iter.**	**UB**	**LB**	**Time (s)**	**# Conflicts**
wab_small_1	**7,956**	600	584	456	234	22	18,746	6,659	600	32,681
wab_small_2	**8,111**	600	644	423	178	21	17,994	7,137	600	32,418
wab_small_3	**9,576**	600	491	421	156	17	19,001	7,617	600	32,908
wab_small_4	**10,757**	600	485	441	108	24	33,430	7,723	600	32,647
wab_small_5	**11,413**	600	489	429	104	17	32,830	8,014	600	35,233
wab_small_6	**10,731**	600	417	435	79	18	27,993	7,724	600	35,054
wab_small_7	**10,509**	600	500	456	138	18	21,340	7,631	600	34,648
wab_small_8	**9,530**	600	523	397	92	18	22,496	7,144	600	35,241
wab_small_9	**9,294**	600	533	376	94	18	23,560	6,325	600	34,994
wab_small_10	**10,525**	600	553	374	141	17	27,073	8,145	600	34,901
wab_small_11	**7,969**	600	490	1,350	230	53	25,752	7,906	600	34,719
wab_small_12	**10,228**	600	472	405	98	16	30,739	7,831	600	34,559
wab_small_13	**9,204**	600	503	403	220	18	20,131	7,290	600	34,852
wab_small_14	**10,309**	600	565	528	189	16	25,833	7,570	600	34,718
wab_small_15	**9,625**	600	486	437	104	22	22,344	7,416	600	31,564
wab_small_16	**9,452**	600	543	396	165	18	19,514	6,771	600	30,956
wab_large_1	22,472	600	257	657	172	18	148,862	42,706	600	104,756
wab_large_2	**44,564**	600	466	646	119	22	164,869	42,223	600	104,289
wab_large_3	**41,463**	600	447	669	143	21	144,652	40,609	600	99,871
wab_large_4	**41,996**	600	481	700	134	21	140,267	40,482	600	99,417
wab_large_5	**42,248**	600	471	680	150	20	146,609	38,484	600	98,649
wab_large_6	19,547	600	261	609	209	15	146,285	40,349	600	98,348
wab_large_7	20,108	600	300	181	212	16	136,442	41,000	600	93,665
wab_large_8	**41,275**	600	468	594	120	21	119,587	39,852	600	93,692
wab_large_9	41,810	600	443	570	84	20	112,732	44,387	600	93,290
wab_large_10	**43,819**	600	471	572	102	14	129,704	43,450	600	93,139
wab_large_11	19,672	600	483	754	236	19	122,636	42,210	600	92,842
wab_large_12	**42,403**	600	469	599	126	21	125,812	41,022	600	92,584
wab_large_13	**38,384**	600	421	559	104	24	112,263	37,150	600	88,088
wab_large_14	**39,238**	600	433	578	99	20	113,559	37,020	600	87,871
wab_large_15	**44,439**	600	450	629	149	24	126,544	44,401	600	92,491
wab_large_16	**41,844**	600	436	603	122	22	137,132	40,684	600	91,412

Table B.11: Results on `smi` instances with a time limit of 1 hour. Bold values indicate that `CD-LBBD` found the better lower bound.

	CD-LBBD						SC-MIP			
Instance	**LB**	**Time (s)**	**# Cuts**	**# Conflicts**	**# Benders iterations**	**# Conflict disc. iter.**	**UB**	**LB**	**Time (s)**	**# Conflicts**
smi_close_0	679	4	5	10	7	4	681	681	5	122
smi_close_1	**4,316**	487	40	64	617	14	4,320	3,011	3,600	752
smi_close_2	1,331	106	29	64	151	9	1,338	1,338	76	659
smi_close_3	1,860	5	6	6	2	2	1,866	1,866	11	554
smi_close_4	24,225	3	3	5	3	3	24,228	24,228	2	10
smi_close_5	314	287	11	7	107	9	315	315	48	1,047
smi_close_6	21,034	7	9	27	48	13	21,040	21,040	6	508
smi_close_7	582	239	29	20	483	11	586	586	880	359
smi_close_8	3,413	8	11	34	18	6	3,419	3,419	11	772
smi_headway_0	1,483	9	7	13	80	4	1,483	1,483	5	154
smi_headway_1	**5,040**	1,171	38	78	1,121	19	5,038	3,669	3,600	5,715
smi_headway_2	2,422	3,600	31	91	273	16	2,888	2,629	3,600	1,989
smi_headway_3	3,552	9	10	12	23	5	3,552	3,552	20	1,312
smi_headway_4	24,797	3	3	5	3	3	24,797	24,797	3	10
smi_headway_5	690	3,600	17	41	28	18	867	703	3,600	3,996
smi_headway_6	22,236	7	10	29	53	16	22,236	22,236	10	1,030
smi_headway_7	1,150	201	31	22	322	10	1,150	1,150	133	644
smi_headway_8	5,159	23	18	70	80	10	5,159	5,159	17	1,266
smi_headway_9	16,006	112	53	109	397	17	16,006	16,006	34	4,002
smi_headway_10	6,219	3,600	351	909	2,435	56	9,007	8,801	3,600	4,188
smi_headway_11	4,312	3,600	25	48	25	10	5,579	5,579	190	3,708

Table B.12: Results on `nor` instances with a time limit of 1 hour. Bold values indicate that `CD-LBBD` found the better lower bound. Empty values mean that the method failed to build the model within the time limit.

	CD-LBBD						SC-MIP			
Instance	**LB**	**Time (s)**	**# Cuts**	**# Conflicts**	**# Benders iterations**	**# Conflict disc. iter.**	**UB**	**LB**	**Time (s)**	**# Conflicts**
nor1_critical_0	4,133	4	12	65	16	14	4,140	4,140	7	1,612
nor1_critical_1	2,416	4	6	66	14	12	2,424	2,424	6	870
nor1_critical_2	3,775	4	11	62	10	12	3,779	3,779	6	970
nor1_critical_3	8,016	8	43	168	46	24	8,024	8,024	13	3,055
nor1_critical_4	1,506	3	2	9	3	5	1,508	1,508	4	75
nor1_critical_5	2,677	3	3	18	7	8	2,679	2,679	4	292
nor1_critical_6	4,491	6	38	78	50	15	4,497	4,497	8	1,514
nor1_critical_7	4,137	4	6	36	11	12	4,145	4,145	6	938
nor1_critical_8	3,836	8	41	84	68	17	3,851	3,851	7	1,043
nor1_critical_9	5,488	5	21	131	33	24	5,495	5,495	8	1,316
nor1_full_0	3,299	57	34	316	50	16	3,310	3,310	220	238,183
nor1_full_1	2,486	53	5	442	12	16	2,489	2,489	195	367,212
nor1_full_2	6,046	15	34	171	40	17	6,053	6,053	23	25,714
nor1_full_3	2,658	59	105	386	147	44	2,664	2,664	67	54,313
nor1_full_4	5,358	45	48	369	58	25	5,377	5,377	214	136,633
nor1_full_5	3,732	50	19	324	25	18	3,743	3,743	1,019	274,305
nor1_full_6	5,262	56	14	469	15	16	5,268	5,268	1,129	354,233
nor1_full_7	1,012	42	7	227	5	6	1,017	1,017	163	350,745
nor1_full_8	5,370	95	61	570	67	29	5,379	5,379	1,389	400,516
nor1_full_9	6,076	56	11	382	17	16	6,085	6,085	197	376,500
nor2_1	4,937	234	330	234	372	27	4,965	4,965	21	12,854
nor2_2	4,619	135	211	240	334	23	4,648	4,648	24	12,592
nor2_3	5,500	64	187	250	208	26	5,522	5,522	18	9,745
nor2_4	6,186	598	620	171	613	53	6,211	6,211	23	8,523
nor2_5	5,416	56	134	249	189	27	5,430	5,430	16	8,523
nor3_1	3,667	335	387	385	495	53	3,701	3,701	17	10,009
nor3_2	5,740	123	180	352	426	48	5,780	5,780	18	10,202
nor3_3	5,562	12	43	225	53	25	5,581	5,581	11	8,792
nor3_4	4,605	28	114	237	120	26	4,617	4,617	13	9,290
nor3_5	2,923	45	110	334	246	45	2,951	2,951	15	9,888
nor4_small_1	**8,972**	164	56	688	35	34	9,015	8,879	3,600	228,651
nor4_small_2	**11,217**	277	85	731	137	40	11,254	10,533	3,600	232,573
nor4_small_3	7,924	150	77	484	46	29	7,948	7,948	1,727	221,013
nor4_small_4	**8,189**	121	45	432	29	20	8,224	8,042	3,600	213,557
nor4_small_5	12,144	208	81	642	86	34	12,161	12,161	1,793	232,018
nor4_large_1	**9,551**	265	86	518	72	27	43,174	7,698	3,600	537,922
nor4_large_2	**8,439**	558	72	1,003	47	38	47,511	6,529	3,600	510,923
nor4_large_3	**11,985**	324	252	600	131	35	12,360	11,042	3,600	225,258
nor4_large_4	**13,262**	1,957	682	906	794	55	28,355	9,788	3,600	368,521
nor4_large_5	**8,043**	226	106	539	58	27	8,067	7,833	3,600	282,689
nor5_small_1	**57,443**	2,728	572	2,622	446	79				
nor5_small_2	**66,657**	3,600	656	1,803	298	78				
nor5_small_3	**57,446**	2,788	523	2,420	404	68				
nor5_small_4	**57,361**	2,890	555	2,478	373	87				
nor5_small_5	**62,744**	3,173	535	2,281	421	74				
nor5_large_1	**72,423**	3,600	675	1,944	299	70				
nor5_large_2	**75,959**	3,600	894	2,933	219	58				
nor5_large_3	**69,679**	3,600	704	2,805	251	76				
nor5_large_4	**69,346**	3,600	660	3,007	223	69				
nor5_large_5	**67,173**	3,600	595	2,411	289	72				

Table B.13: Results on `swi` instances with a time limit of 1 hour. Bold values indicate that `CD-LBBD` found the better lower bound.

	CD-LBBD						SC-MIP			
Instance	**LB**	**Time (s)**	**# Cuts**	**# Conflicts**	**# Benders iterations**	**# Conflict disc. iter.**	**UB**	**LB**	**Time (s)**	**# Conflicts**
swi_1	0	3	0	-	0	1	0	0	3	266
swi_2	0	7	0	505	0	4	0	0	12	15,269
swi_3	0	13	0	1,226	0	7	0	0	38	48,965
swi_4	0	16	0	1,474	0	8	0	0	45	56,030
swi_5	0	16	0	1,462	0	8	0	0	44	56,031
swi_6	0	54	0	3,739	0	10	0	0	163	148,452
swi_7	0	76	0	6,172	0	10	0	0	262	260,791
swi_8	0	44	0	2,031	0	24	0	0	187	115,831
swi_9	0	594	0	14,097	0	192	0	0	414	75,594

Table B.14: Results on `wab` instances with a time limit of 1 hour. Bold values indicate that `CD-LBBD` found the better lower bound.

	CD-LBBD						SC-MIP			
Instance	**LB**	**Time (s)**	**# Cuts**	**# Conflicts**	**# Benders iterations**	**# Conflict disc. iter.**	**UB**	**LB**	**Time (s)**	**# Conflicts**
wab_small_1	**9,894**	3,600	944	672	298	53	21,119	8,537	3,600	32,681
wab_small_2	**9,560**	3,600	1,059	781	334	55	18,793	8,749	3,600	32,418
wab_small_3	**11,440**	3,600	851	655	232	49	19,917	9,179	3,600	32,908
wab_small_4	**11,615**	3,600	930	708	192	54	22,607	9,605	3,600	32,647
wab_small_5	**12,862**	3,600	973	702	207	50	24,915	10,182	3,600	35,233
wab_small_6	**12,605**	3,600	808	852	142	47	21,632	9,436	3,600	35,054
wab_small_7	**12,558**	3,600	834	863	220	48	21,572	10,027	3,600	34,648
wab_small_8	**11,485**	3,600	891	620	149	49	21,304	9,532	3,600	35,241
wab_small_9	**10,427**	3,600	922	772	163	54	23,074	8,170	3,600	34,994
wab_small_10	**12,942**	3,600	943	674	181	46	24,877	9,741	3,600	34,901
wab_small_11	**12,598**	3,600	931	652	209	47	23,579	9,520	3,600	34,719
wab_small_12	**12,162**	3,600	883	674	147	44	22,557	9,437	3,600	34,559
wab_small_13	**11,301**	3,600	939	784	338	52	19,856	9,182	3,600	34,852
wab_small_14	**11,068**	3,600	918	643	225	47	19,532	9,445	3,600	34,718
wab_small_15	**11,909**	3,600	998	759	222	55	22,994	9,922	3,600	31,564
wab_small_16	**11,677**	3,600	788	922	161	59	16,090	8,167	3,600	30,956
wab_large_1	45,789	3,600	596	753	214	39	129,289	46,270	3,600	104,756
wab_large_2	46,379	3,600	591	745	117	29	144,324	47,616	3,600	104,289
wab_large_3	44,955	3,600	567	775	138	30	129,241	45,092	3,600	99,871
wab_large_4	44,145	3,600	566	782	124	29	150,694	44,660	3,600	99,417
wab_large_5	**44,924**	3,600	615	763	162	28	131,238	38,453	3,600	98,649
wab_large_6	**45,716**	3,600	637	764		26	128,161	44,469	3,600	98,348
wab_large_7	**45,199**	3,600	657	789	205	29	125,082	45,116	3,600	97,665
wab_large_8	**44,523**	3,600	571	675	121	31	106,498	43,446	3,600	93,692
wab_large_9	**45,107**	3,600	577	663	86	27	117,674	44,387	3,600	93,290
wab_large_10	45,899	3,600	573	672	112	24	119,841	46,248	3,600	93,139
wab_large_11	**46,382**	3,600	557	664	106	28	117,077	45,786	3,600	92,842
wab_large_12	45,373	3,600	614	729	116	32	111,613	45,609	3,600	92,584
wab_large_13	**42,018**	3,600	543	628	102	32	111,539	40,969	3,600	88,088
wab_large_14	**40,785**	3,600	546	618	103	34	117,243	40,753	3,600	87,871
wab_large_15	46,085	3,600	586	690	152	32	111,023	47,882	3,600	92,491
wab_large_16	**44,214**	3,600	560	673	122	30	119,814	43,835	3,600	91,412